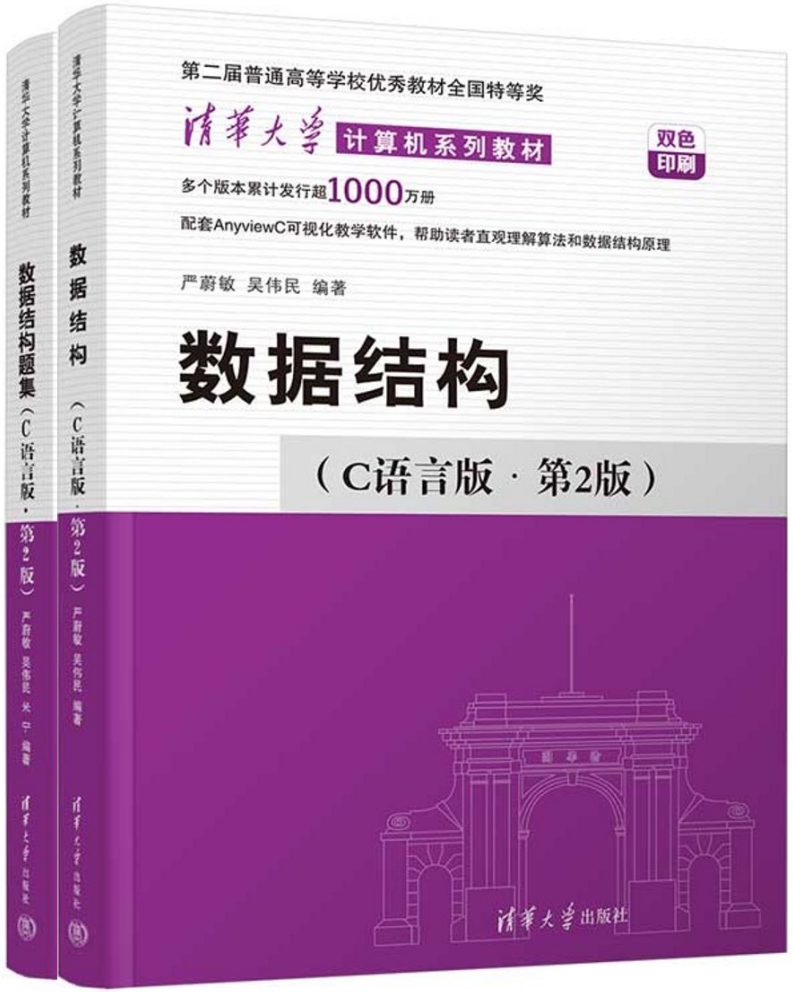


清華大学出版社
TSINGHUA UNIVERSITY PRESS

第9章

查 找

清華大學出版社 TSINGHUA UNIVERSITY PRESS

查找表（Search Table）是由同一类型的数据元素（或记录）构成的集合。“集合”中的数据元素之间没有约定任何关系，是一种非常灵便的数据结构，可以根据需要对数据元素之间附加某种关系和设计特别的存储结构。查找表的基本操作有：

（1）**查询**（Contain）某个数据元素是否存在，因而也称为**查存**；

（2）**检索**（Retrieval）某个数据元素的数据项；

（3）**插入**（Insert）一个数据元素；

（4）**删除**（Delete）某个数据元素。

静态查找表（Static Search Table）——只作前两种统称为“**查找**（Search）”的操作。

动态查找表（Dynamic Search Table）——可基于查找结果，**插入**查找表中不存在的数据元素，或者从表中**删除**已存在的某个数据元素。

典型的关键字类型说明可以是下列几种之一

```
typedef float    KeyType;     // 实型
typedef int      KeyType;     // 整型
typedef char    *KeyType;     // 字符串型
```

数据元素类型定义为：

```
typedef struct {
  KeyType key;   // 关键字（简称键）域
  …      // 其他域,如值域：ValueType value;
} SElemType;    // 元素类型，可沿用ElemType
```

结构型元素之间的比较实际上是关键字的比较。需要为其定义专门的比较函数。例如SElemType类型元素的“小于”比较函数，可如下定义（假设KeyType是可直接比较的类型），其余比较函数就不一一列举了。

```
Status LT(SElemType a, SElemType b)
  { return a.key < b.key; }
```



9.1 静态查找表

清華大學出版社 TSINGHUA UNIVERSITY PRESS

抽象数据类型静态查找表的定义为：

ADT StaticSearchTable {

数据对象D： D是具有相同特性的数据元素集合。各数据元素均含有类型相同且可唯一标识数据元素的关键字。

数据关系R： 数据元素同属一个集合。

基本操作P：

MakeST(es, n);

初始条件：n>0，es含有n个数据元素。

操作结果：由es前 n个数据元素构造并返回一个静态查找表。

FreeST(ST);

初始条件：静态查找表ST存在。

操作结果：回收表ST，返回NULL。

Search(ST, key);

初始条件：静态查找表ST存在，key为和关键字类型相同的给定值。

操作结果：若ST中存在关键字等于key的数据元素，则返回该元素值或在表中位置，否则返回nullE。

Traverse(ST, Visit());

初始条件：静态查找表ST存在，Visit是对元素操作的应用函数。

操作结果：按某种次序对ST每个元素调用函数Visit()一次且仅一次。若Visit()失败，则操作失败。

} ADT StaticSearchTable

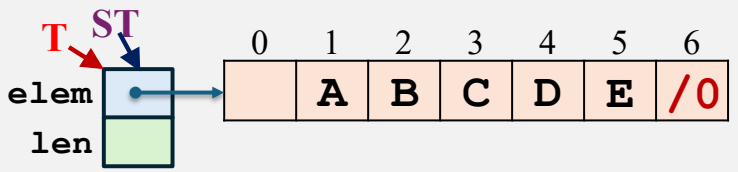
静态查找表可以有不同的表示方法，在不同的表示方法中，实现查找操作的方法也 不同。

顺序表类型和构造

```
//----- 静态查找表的顺序存储结构及构造函数 -----
typedef struct {
   ElemType  *elem;     // 数据元素存储空间基址，建表时按实际长度分配，0号单元留空
   int        len;      // 表长度
} *SSTable;             // 静态查找表指针类型
SSTable MakeST(ElemType *es, int n) {  // 由es前 n个数据元素构造并返回一个静态查找表
   SSTable ST;
   if (!(ST = (SSTable)malloc(sizeof(*ST)))) exit(OVERFLOW);    // 查找表结构记录
   if (!(ST->elem = (ElemType*)malloc((n+2)*sizeof(ElemType)))) exit(OVERFLOW);
   memcpy(ST->elem+1, es, n*sizeof(ElemType));  // 复制es前n个元素值到查找表的元素区
   ST->len = n;
   return ST;
}
```

构造过程示例（假设元素为字符类型）

```
ElemType s = "ABCDE";
SSTable T = MakeST(s, 5);
```



9.1.1 顺序表的查找

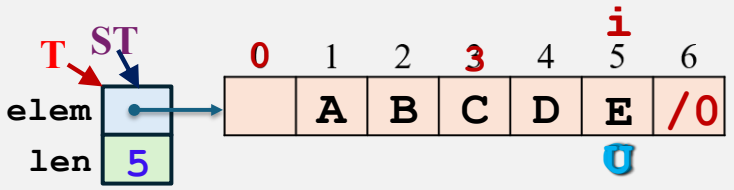
```
int Search_Seq(SSTable ST, KeyType key) {  // 算法9.1
    // 在静态查找表ST顺序查找关键字等于key的数据元素，找到则返回其位序，否则返回0
    ST->elem[0].key = key;                                  //  “哨兵”
    for (int i=ST->len;  ST->elem[i].key!=key;  --i);   // 从后往前找
    return i;                                                   // 找不到时，i为0
}
```

这个算法的思想和第2章中的函数LocateElem_Sq一致。只是在Search_Seq中，查找之前先对ST->elem[0]的关键字赋值key，目的在于免去查找过程中每一步都要检测整个表是否查找完毕。在此，ST->elem[0]起到了监视哨的作用。

这仅是一个程序设计技巧 上的改进，然而实践证明，这个改进能使顺序查找在ST->len≥1000时，进行一次查找所需的平均时间几乎减少一半。当然，监视哨也可设在高下标处。

查找示例1

```
int loc = Search_Seq(T, 'C');
```



查找示例2

```
int loc = Search_Seq(T, 'U');
```

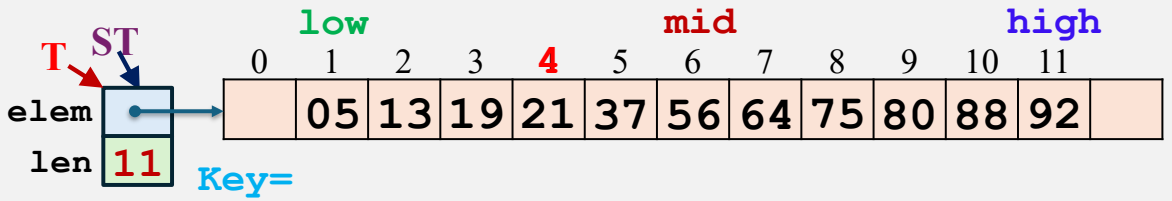
未找到

9.1.2 有序表的查找

```
int Search_Bin(SSTable ST, KeyType key) {   // 算法 9.2
    // 在有序表ST中折半查找关键字等于key的数据元素，找到则返回该元素的位序，否则返回0
    int low = 1, high = ST->len;          // 置区间初值
    while (low <= high) {                 // 查找区间非空的循环
        int mid = (low + high) / 2;       // 中间位序
        if (key == ST->elem[mid].key) return mid;  // 找到待查元素，返回其位序
        else if (key < ST->elem[mid].key) high = mid-1;   // 继续在前半区间进行查找
        else low = mid + 1;                                // 继续在后半区间进行查找
    }
    return 0;   // 不存在查找的元素，返回0
}
```

在有序表 T 查找 21

```
int loc = Search_Bin(T, 21);
```



折半查找的性能分析

查找21的过程可用图9.2所示的二叉树来描述。树中每个节点表示表中一个记录，节点中的值为该记录在表中的位置，节点上方是表中该位置的元素值。通常称这个描述查找 过程的二叉树为判定树。

折半查找法在查找成功时进行比较的关键字个数最多不超过树的深度,而具有n个节点的判定树的深度为$\lfloor \log_2 n \rfloor+1$，所以，折半查找法在查找成功时和给定值进行比较的关键字个数至多为$\lfloor \log_2 n \rfloor+1$ 。

在图9.2所示判定树中所有节点的空指针域 上加一个指向 一 个方形节点的指针，如图 9.3所示。并且，称这些方形节点为判定树的外部节点 (与之相对，称那些圆形节点为内部节点)，那么折半查找时查找不成功的过程就是走了一条从根节点到外部节点的路径，和给定值进行比较的关键字个数等于该路径上内部节点个数，例如，查找85的过程即为走了一条从根到节点9-10的路径。因此，折半查找在查找不成功时和给定值进行比较的关键字个数最多也不超过$\lfloor \log_2 n \rfloor+1$。

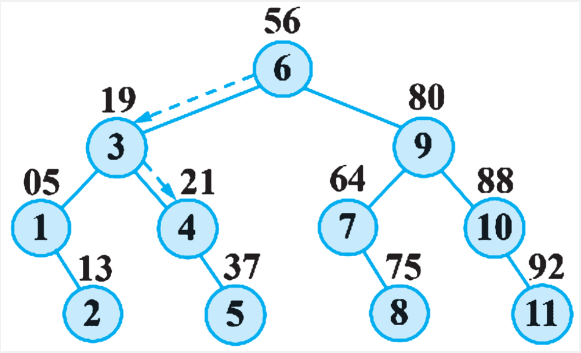


图9. 2　描述折半查找过程的判定树及查找21的过程

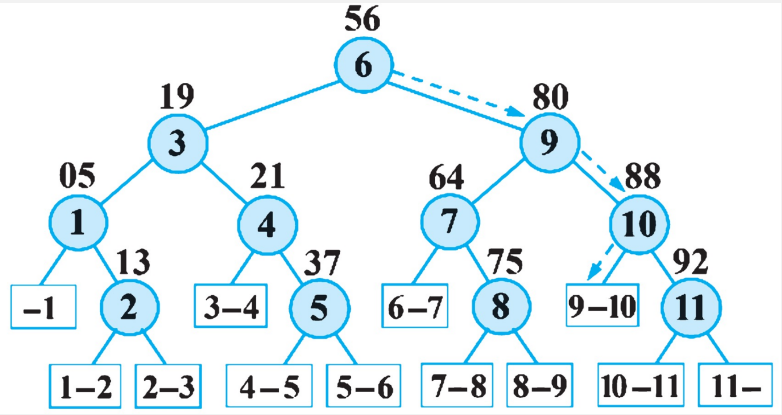
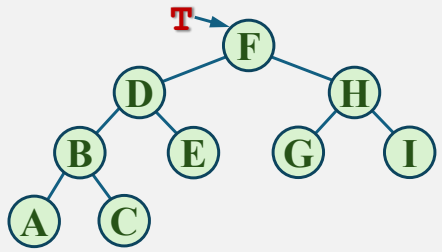


图9.3　加上外部节点的判定树和查找85的过程

构造次优查找树

清華大學出版社 TSINGHUA UNIVERSITY PRESS



j	0	1	2	3	4	5	6	7	8	9
key_j		A	B	C	D	E	F	G	H	I
W_j	0	1	1	2	5	3	4	4	3	5
SW_j	0	1	2	4	9	12	16	20	23	28

图9.4 算法9.4和9.3构造次优二叉查找树示例

```
typedef  BiTree  SOSTree;  // 次优查找树采用二叉链表
SOSTree CreateSOSTree(SSTable ST) {
  // 算法9.4。由有序表ST构造次优查找树T。ST的元素含权值
  SOSTree T;   float *sw;
  if (ST->len == 0) T = NULL;  // 构造空树T
  else {
    if (!(sw= (float*)malloc(ST->len*sizeof(float))))
      exit(OVERRFLOW);
    sw[0]=0;
    for(int j=1; j<=ST->len; j++)
      sw[j] = sw[j-1] + ST->elem[j].weight;
    T = SecondOptimal(ST->elem, sw, 1, ST->len);
  }       // 调用算法9.3
  return T;
} // CreateSOSTree
```

构造次优查找树

清華大學出版社 TSINGHUA UNIVERSITY PRESS

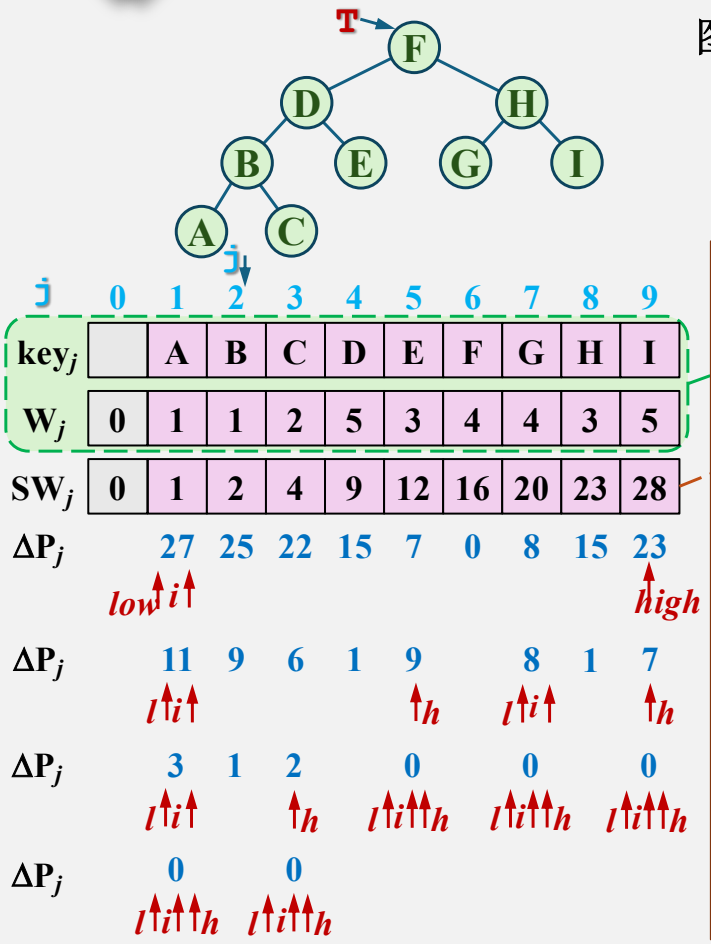


图9.4 算法9.4和9.3构造次优二叉查找树示例

```
BiTree SecondOptimal(ElemType R[], float sw[],
BiTree SecondOptimal(ElemType R[], float sw[],
BiTree SecondOptimal(ElemType R[], float sw[],
                     int low, int high) {  // 算法9.3
  // 由有序表R[low..high]及其累计权值表sw递归构造次优查找树T。
  int i = low;  float min = (float)fabs(sw[high]-sw[low]);
  float dw = sw[high]+sw[low-1];
  for (int j=low+1;  j<=high;  ++j)          // 选择最小的ΔPi值
    if (fabs(dw-sw[j]-sw[j-1]) < min)
      { i = j;  min = (float)fabs(dw-sw[j]-sw[j-1]); }
  BiTree T;
  if (!(T=(BiTree)malloc(sizeof(BiTNode)))) exit(OVERFLOW);
  T->data = R[i];       // 生成结点
  if (i==low) T->lchild = NULL;                    // 左子树空
  else T->lchild = SecondOptimal(R,sw,low,i-1);    // 建左子树
  if (i==high) T->rchild = NULL;                   // 右子树空
  else T->rchild = SecondOptimal(R,sw,i+1,high);   // 建右子树
  return T;
} // SecondOptimal
```

二叉排序树的查找

清華大學出版社 TSINGHUA UNIVERSITY PRESS

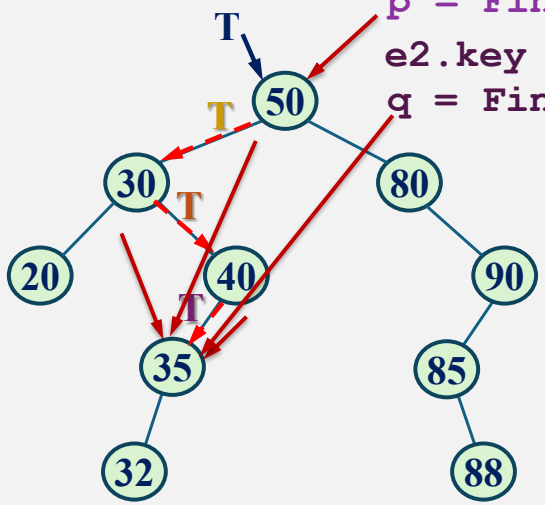
算法9.6 在根指针T所指二叉排序树中递归地查找其关键字等于key的数据元素

若查找成功，则指针p指向该数据元素结点，并返回TRUE，

否则指针p指向查找路径上访问的最后一个结点并返回FALSE，

指针f指向T的双亲，其初始调用值为NULL。

查找：
```
e1.key = 50;
p = Find(T, e1);
e2.key = 35;
q = Find(T, e2);
```

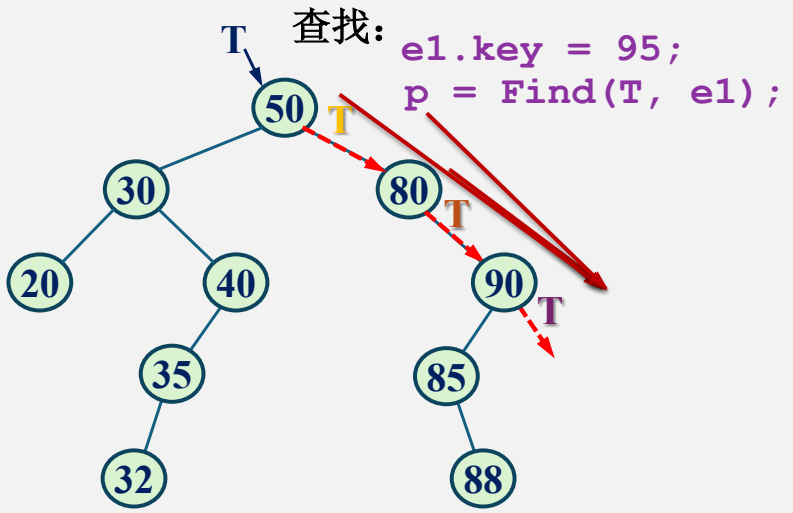



```
BiTNode *Find(BiTree T, ElemType e) { // 算法9.6
  if (T==NULL) return NULL;                          // 空树，查找不成功
  if (LT(e, T->data)) return Find(T->lchild, e);     // 返回在左子树查找结果
  else if (GREAT(e, T->data)) return Find(T->rchild, e); // 返回在右子树查找结果
       else return T;
}
```

二叉排序树的查找（续）

清華大學出版社 TSINGHUA UNIVERSITY PRESS

算法9.6 在根指针T所指二叉排序树中递归地查找其关键字等于key的数据元素，
若查找成功，则指针p指向该数据元素结点，并返回TRUE，
否则指针p指向查找路径上访问的最后一个结点并返回FALSE，
指针f指向T的双亲，其初始调用值为NULL。




```
BiTNode *Find(BiTree T, ElemType e) { // 算法9.6
  if (T==NULL) return NULL;                          // 空树，查找不成功
  if (LT(e, T->data)) return Find(T->lchild, e);     // 返回在左子树查找结果
  else if (GREAT(e, T->data)) return Find(T->rchild, e); // 返回在右子树查找结果
       else return T;                                // 找到，返回节点指针
}
```

二叉排序树的插入

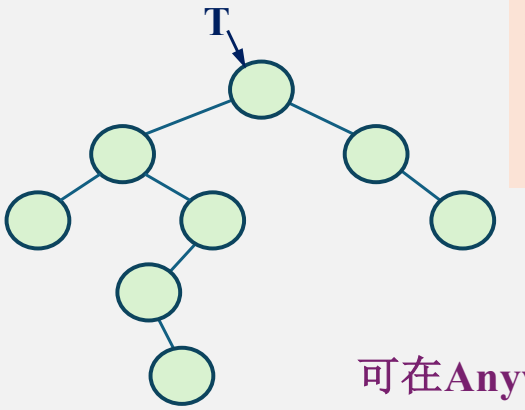
清華大學出版社 TSINGHUA UNIVERSITY PRESS

算法9.7 当二叉排序树T中不存在关键字等于e.key的数据元素时，插入e并返回T。

首先观察：从空树T开始，依次插入以下序列的过程。

```
BiTree InsertBST(BiTree T, ElemType e) {  // 算法9.7
  if (T==NULL) { //
    if (!(T = (BiTNode*)calloc(sizeof(BiTNode)))) exit(OVERFLOW);
    T->data = e;  T->lchild = T->rchild = NULL;  return T;   // 生成结点
  }
  if (LT(e,p->data)) T->lchild = InsertBST(T->lchild,e);      // 插入左子树
  else if (GT(e,T->data) T->rchild = InsertBST(T->rchild,e); // 插入右子树
       // 否则，树中已有关键字相同的结点，不再插入
  return T;  // 必须返回T，而且赋给T的实参——T可能改变
}
```

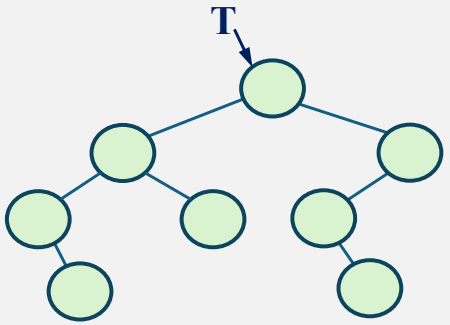
50, 30, 40, 80, 20, 35, 90, 40, 38



再观察：不同的插入顺序

小结：
（1）不含重复关键字
（2）总是插入为叶子
（3）树形与插入次序相关

40, 90, 35, 50, 20, 38, 80, 40, 30



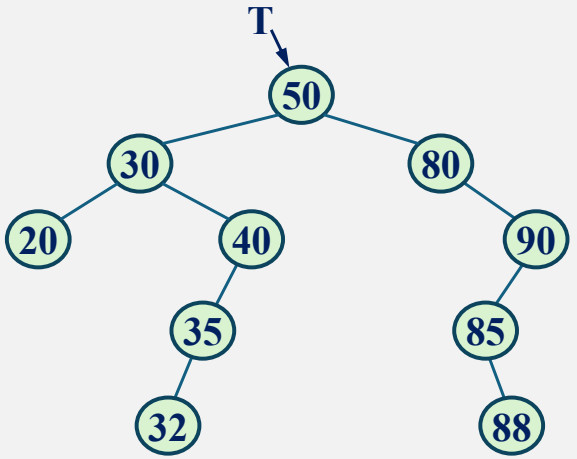
可在AnyviewC可视跟踪观察算法过程中二叉排序树的形态变化

二叉排序树的删除

例如删除：20　88

观察三种情形的删除：

（1）被删除的结点是**叶子结点**



其双亲结点中相应指针域的值改为“空”

清華大學出版社
TSINGHUA UNIVERSITY PRESS

二叉排序树的删除（续）

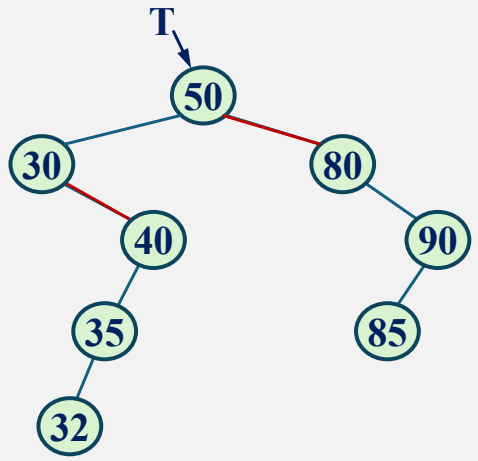
例如删除： 40 (其左子树非空)

80 (其右子树非空)

观察三种情形的删除：

（1）被删除的结点是**叶子结点**

（2）被删除的结点的**左、右子树只有一棵非空**



其双亲结点的相应指针域的值改为：“指向被删除结点的左子树或右子树”。

二叉排序树的删除（续）

清華大學出版社 TSINGHUA UNIVERSITY PRESS

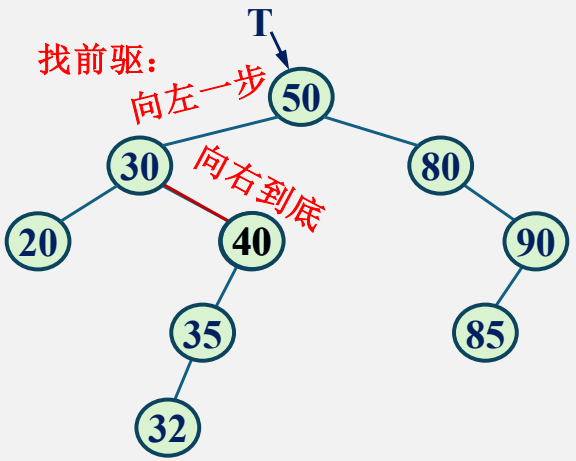
例如删除： 50 （其左、右子树均不空）

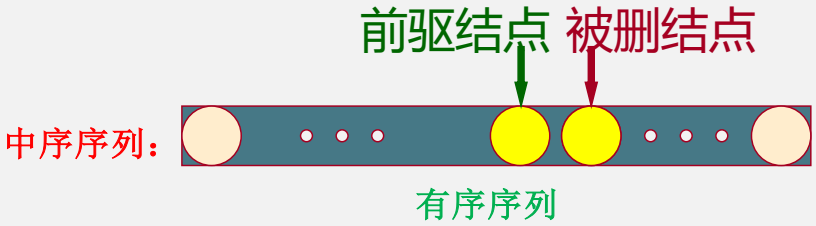
观察三种情形的删除：

（1）被删除的结点是**叶子结点**

（2）被删除的结点的**左、右子树只有一棵非空**

（3）被删除的结点的**左、右子树都不空**





以其前驱替代之，然后再删除该前驱结点

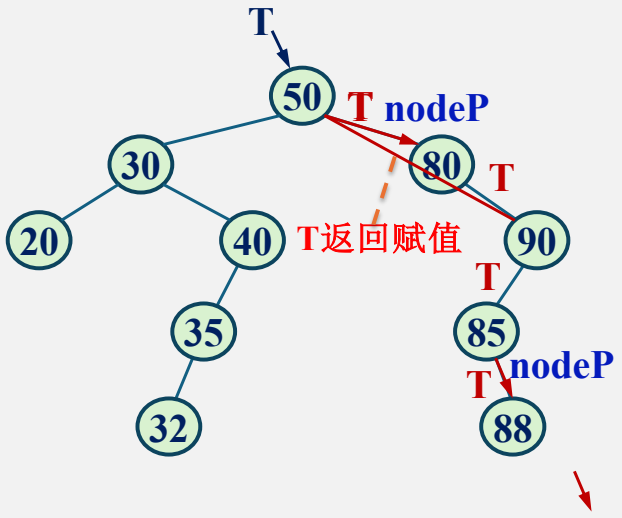
前驱结点的右子树为空！

二叉排序树的删除（续）

清華大學出版社 TSINGHUA UNIVERSITY PRESS

对前面观察的三种情形，
分别进行算法9.8示例演示。

例如删除： 88（叶子） 80（左子树为空）
经若干递归：




```
BiTree DeleteDST(BiTree T, ElemType e) { // 算法9.8 在T中删除元素e并返回树根指针
  BiTNode *nodeP, *f;
  if (T == NULL) return NULL;                          // 找不到e，则返回NULL
  if (LT(e,T->data)) T->lchild = DeleteDST(T->lchild, e); // 若e小则对左子树递归
  else if (GT(e,T->data)) T->rchild = DeleteDST(T->rchild, e); //e大则对右子树递归
  else {                                               // 到此，找到了e，分两种情形删除e
    if (T->lchild && T->rchild) {                      // 若T节点的左右孩子齐全 （3）
      f = T;  nodeP = FindMin(T->rchild, &f);          // 找右子树的最小元素顶替e（做法二）
      if (f->lchild == nodeP) f->lchild = nodeP->lchild;   // 是f节点的左孩子
      else f->rchild = nodeP->lchild;                  // 否则是右孩子
      T->data = nodeP->data;   free(nodeP);            // 最小元素顶替被删元素且实际删除
    } else {                                           // 否则，只有1个或0个孩子 （1）和（2）
      nodeP = T;
      if (T->lchild == NULL) T = T->rchild;            // 无左孩子则选右子树(也处理了0个孩子)
      else if(T->rchild == NULL)  T = T->lchild;       // 否则选左子树(返回接为子树)
      free(nodeP);                                     // 释放实际被删节点
    }
  }
  return T;    // 返回T（可能被改变了）
}
```

之后逐层返回

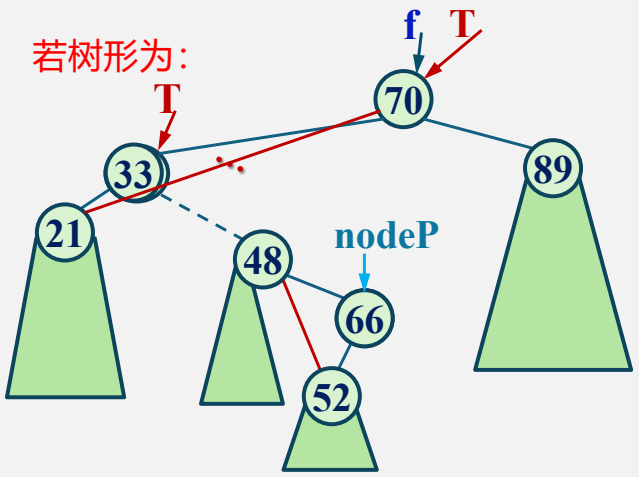
二叉排序树的删除（续）

清華大學出版社 TSINGHUA UNIVERSITY PRESS

对前面观察的三种情形，
分别进行算法9.8示例演示。

例如删除： 70（其左、右子树均不空）
经若干递归：

若树形为：




```
BiTree DeleteDST(BiTree T, ElemType e) { // 算法9.8 在T中删除元素e并返回树根指针
  BiTNode *nodeP, *f;
  if (T == NULL) return NULL;                          // 找不到e，则返回NULL
  if (LT(e,T->data)) T->lchild = DeleteDST(T->lchild, e); // 若e小则对左子树递归
  else if (GT(e,T->data)) T->rchild = DeleteDST(T->rchild, e); //e大则对右子树递归
  else {                                  // 到此，找到了e，分两种情形删除e
    if (T->lchild && T->rchild) {         // 若T节点的左右孩子齐全 （3）
      f = T;   nodeP = FindMax(T->lchild, &f);    // 找左子树的最大元素顶替树中的e
      if (f->lchild == nodeP) f->lchild = nodeP->lchild;    // 是f节点的左孩子
      else f->rchild = nodeP->lchild;                       // 否则是右孩子
      T->data = nodeP->data;   free(nodeP);   // 最小元素顶替被删元素且实际删除
    } else {                              // 否则，只有1个或0个孩子 （1）和（2）
      nodeP = T;
                  传递f的地址  *p即f
BiTNode *FindMin(BiTree T, BiTNode **p) { // 迭代查找最大元素，返回其节点指针
  if (T != NULL)                          // T空则不存在，返回NULL
    while (T->rchild != NULL) { *p = T;   T = T->rchild; }     // 向右到底
  return T;                               // 返回最右下的节点指针
}                                                  算法9.5中的函数
  return T;    // 返回T（可能被改变了）
}
```

9.2.2 平衡二叉树

1. 平衡二叉树定义

平衡二叉树（Balanced Binary Tree 或 Height-Balanced Tree）又称**AVL树**（取自两位发明者的名字Adelson-Velskii和Landis）是带有平衡条件的二叉排序树。它或者是一棵空树，或者是具有下列性质的二叉排序树：它的左子树和右子树的都是平衡二叉树，且左、右子树的深度之差不超过1。为了避免频繁求任一子树的高度，在每个节点增设一个域，存放以该节点为根的子树的高度或平衡性相关的值，主要分两种：

1. **平衡因子**（Balanced Factor），定义为该节点的左子树的深度减去它的右子树的深度。平衡二叉树上所有节点的平衡因子只能是-1、0和1。只要有一个节点的平衡因子绝对值大于1，则该二叉树就是不平衡的。
2. **树高度**（Height of Tree），定义为以该节点为根的子树的高度（空树高度为0）。平衡二叉树上所有节点的左右子树的高度之差不超过1（树的**高度**和**深度**是同义词）。

下面基于在节点增设高度域h展开对AVL树的讨论，而设平衡因子的情形留给读者思考和练习。

AVL树的存储表示定义如下（在二叉链表基础上增设高度域h），为讨论和代码简洁，直接用关键字代表数据元素，实际应用可根据需要增设其他数据域。

```
typedef struct Avlnode {
  KeyType  key;     // 关键字域
  Avlnode *lchild, // 左孩子指针域
          *rchild; // 右孩子指针域
  int   h;   // 保存本节点为根的子树的高度
} AvlNode, *AvlTree; // AVL树指针类型
```

我们希望由任何初始序列插入构成的二叉排序树都是AVL树。AVL树上任何节点的左、右子树的高度之差不大于1，可以证明它的高度和log*n*是同数量级的（其中*n*为节点个数）。因此它的平均查找长度也和log*n*同数量级。

9.2.2 平衡二叉树（续）

2. 平衡二叉树的插入

AVL树的插入操作是在二叉排序树插入操基础上，需要维护通向根节点路径上的节点平衡性。一旦发生这种情况，则称该节点**失衡**，恢复平衡后才认为本次插入完成。事实上，这总可以通过对树进行称为**旋转**（rotation)的简单修正做到。

在插入以后，可能破坏了父节点的平衡性，而不会影响其兄弟节点。只有那些从插入点到根节点的路径上的节点可能失衡，因为只有这些节点的子树可能发生变化。下面讨论在向上到根的路径上，对第一个这样的节点（即最深的节点）为根的子树如何维护平衡性，并证明这一重新平衡保证整棵树满足AVL特性。

把必须重新平衡的节点称作α。由于任一节点最多有两个孩子，因此高度不平衡时，α节点的两棵子树的高度差为2。这种不平衡可能出现在下面四种情况之中：

1.在α的左孩子的左子树插入一个节点，称为**LL型**插入；

2.在α的左孩子的右子树插入一个节点，称为**LR型**插入；

3.在α的右孩子的左子树插入一个节点，称为**RL型**插入；

4.在α的右孩子的右子树插入一个节点，称为**RR型**插入。

LL和RR型插入是关于α点的镜像对称，而情形LR和RL型插入是关于α点的镜像对称。因此，理论上只有两种情形，当然从编程的角度来看还是四种情形。

第一种情形（即LL型或RR型）是插入发生在树的“外侧”的情形，通过对树一次**单旋转**（single rotation）可完成调整。

第二种情形（即LR型或RL型）是插入发生在树的“内侧”的情形，需要**双旋转**（double rotation）处理。

这些旋转是对树的基本操作，也用于另一种平衡树——红黑树。

2. 平衡二叉树的插入（续）

（1）单旋转

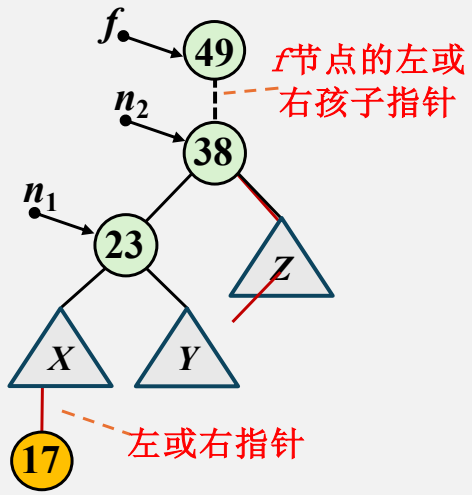



```
AvlNode* R_Rotate(AvlNode *n2) {   // 算法 9.9，对n2子树右单旋（对LL失衡的调整）
    AvlNode *n1 = n2->lchild;                  // *n1是*n2的左孩子（肯定不空）
    AvlNode *lrc = n1->rchild;                 // *lrc是*n1的右孩子（可能为空）
    n1->rchild = n2;     n2->lchild = lrc;     // 右旋操作
    updateHeight(n2);    updateHeight(n1);     // 更新n2和n1节点的高度值
    return n1;                                 // 返回新根节点指针
}
```

可见修改了两个指针：49的左孩指针和23的右孩指针。可把这个具体示例提炼为LL型插入的再平衡处理模式：

- ✧将作为处理焦点的“三代”节点49（祖）、38（子）和23（孙）一般化，分别用f、n_2和n_1指针指向它们；
- ✧模糊化49，相比38它可大可小，即38可以是其左孩子也可以是右孩子；
- ✧考虑38的右子树Z和23的左子树X与右子树Y，以及由算法9.9进行旋转处理时对相关指针的相应操作。

从对LL型模式的一般化处理和结果可见，共改变了三个相关指针。



2. 平衡二叉树的插入（续）

（2）双旋转

如图9.14(a)和(b)所示，在n_3节点的左（右）孩子的右（左）子树Y中插入一个节点（LR和RL型插入）导致n_3子树失衡，仅用一次右旋或左旋无法实现再平衡。原因是子树Y太高，单次旋转未能降低它的高度。

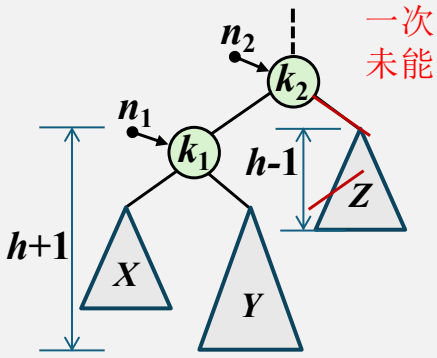
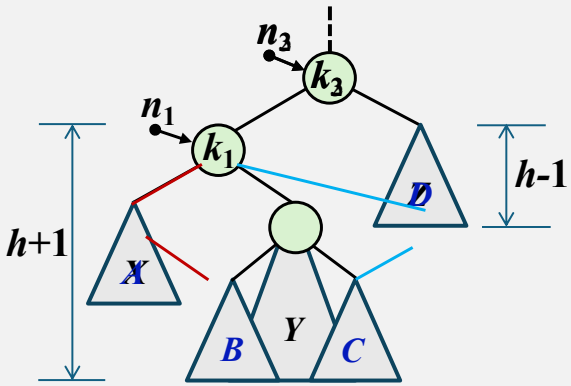


图9.14(c)展示了解决办法——双旋转。

算法9.10先后调用左旋和右旋，实现了LR型的双旋转。

```
Position L_R_Rotate(AvlNode *n3) {   // 算法 9.10
    // LR型插入后失衡的左子树双旋转再平衡处理
    // 调用条件：n3节点有左孩子且其左孩子有右孩子
    n3->lchild = L_Rotate(n3->lchild); // 左旋
                 // n3节点(k3)的左孩子k2与其右孩子k1左旋
    return R_Rotate(n3);        // k3与k1右旋
}
```



算法9.11实现AVL树的插入

二叉排序树插入 + 再平衡

```
AvlTree AvlInsert(AvlTree T, KeyType key) {   // 算法9.11
   if (T == NULL) {   // 若T空，则构建并返回一棵单节点树(若是递归到底，则返回新节点接为叶子)
      if (!(T = (AvlNode*)malloc(sizeof(AvlNode)))) exit(OVERFLOW); // 分配结点
      T->key = key;    T->h = 1;    T->lchild = T->rchild = NULL;      // 高度为1
      return T;                                                       // 返回T
   }
   if (key < T->key) {                              // 若key小于T元素
      T->lchild = AvlInsert(T->lchild, key); // 则向左递归，插入到左子树后返回其根指针
      if (T->lchild->h - T->rchild->h == 2)   // 递归插入返回后，若高差等于2
         if (key < T->lchild->key)            // 且若key小于左孩子（插入在左孩的左子树）
            T = R_Rotate(T);                  // 则属LL型，作右旋
         else T = L_R_Rotate(T);              // 否则属LR型，对左子树双旋再平衡处理
   } else if(key > T->key) {                  // 若key大于T元素
      T->rchild = AvlInsert(T->rchild, key); // 则操作代码与“小于”情形左右对称
      if (T->rchild->h - T->lchild->h == 2)   // 递归插入返回后，若高差等于2
         if (key > T->rchild->key)            // 大于
            T = L_Rotate(T);                  // 属RR型，左旋
         else  T = R_L_Rotate(T);             // 属RL型，对右子树双旋再平衡处理
   }                                          // 否则，key已在树中，不重复插入
   updateHeight(T);                           // 更新高度
   return T;                                  // 返回T（树根T可能被改变了）
}
```

3. 平衡二叉树的删除

和插入类似，AVL树的删除也是以二叉排序树的递归删除为基础，确保有序性。

```
AvlTree AvlDelete(AvlNode *T, KeyType key) {   // 算法9.12，AVL树删除操作
   if (!T) return NULL;
   if (key < T->key) {                                  // 1. 标准BST删除过程
      T->lchild = AvlDelete(T->lchild, key);
   } else if (key > T->key) {
      T->rchild = AvlDelete(T->rchild, key);
   } else {                                                          // 找到了要删除的节点
      if (!T->lchild || !T->rchild) {                                // 若是叶子或有单孩，则
         AvlNode *temp = T->lchild ? T->lchild : T->rchild;          // 取该孩指针（叶则NULL）
         free(T);   return temp;                                     // 回收节点，返回孩子指针
      } else {                                                       // 否则有双孩
         AvlNode *succ = findSuccessor(T->rchild);                   // 求其后继节点
         T->key = succ->key;                                         // 用后继关键字顶替
         T->rchild = AvlDelete(T->rchild, succ->key);                // 递归删除后继节点
      }
   }
   updateHeight(T);                                     // 2. 更新高度
   return Balance(T);                                   // 3. 维护平衡并返回根指针
}
```

3. 平衡二叉树的删除（续）

清華大學出版社 TSINGHUA UNIVERSITY PRESS

在递归返回时向上检测并维护树的平衡性。

```
int getBalanceFactor(AvlNode *node)   // 算法 9.13，由左右子树高度值计算节点的平衡因子
   { return getHeight(node->lchild) - getHeight(node->rchild); }

AvlNode* balance(AvlNode *node) {     // 删除节点后的维护平衡（用高度差评判平衡性）
   int bf = getBalanceFactor(node);   // 求节点的平衡因子
   if (bf > 1)                        // 若左子树失衡，则按失衡类型作相应单旋或者双旋处理
      if (getBalanceFactor(node->lchild) >= 0)       // 若左孩的左子树更高
         return (node->lchild->lchild) ? R_Rotate(node) : L_R_Rotate(node);
      else
         return (node->lchild->rchild) ? L_R_Rotate(node) : L_Rotate(node);
   if (bf < -1)                       // 若右子树失衡，则按失衡类型作相应单旋或者双旋处理
      if (getBalanceFactor(node->rchild) <= 0)       // 若右孩的右子树更高
         return (node->rchild->rchild) ? L_Rotate(node) : R_L_Rotate(node);
      else
         return (node->rchild->lchild) ? R_L_Rotate(node) : R_Rotate(node);
   return node;                                      // 返回结点指针（可能改变）
}
```

可在AnyviewC可视跟踪观察算法过程中平衡二叉树的形态变化

9.2.3 红黑树

1. 红黑树的性质

红黑树（Red-black Tree）也是一种平衡二叉排序树，因其统计性能好于AVL树，而受到关注和更多应用。具有下列性质的二叉排序树是红黑树：

①每个节点要么是红的，要么是黑的；

②根节点是黑的；

③所有叶节点都是空节点（nil node），并且是黑的；

④红节点的两个子节点都是黑的；

⑤节点到其每个叶子节点的路径都包含相同数目的黑节点。

命名红黑树的原因在于它的每个节点都被“着色”为红色或黑色。节点的颜色被用来检测树的平衡性。

从某节点x出发（不包括该节点）到达一个叶节点的路径上黑节点个数被称为该节点的**黑高度**（Black-height），用bh(x)表示。性质⑤确保任一节点的黑高度是确定的。红黑树的黑高度定义为根节点的黑高度。红黑树的性质保证了从根节点到叶节点的路径长度不会超过任何其他路径的两倍。

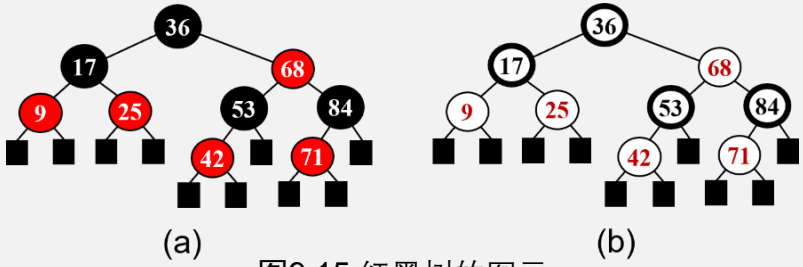


图9.15 红黑树的图示

引理9.1 一棵有n个内节点的红黑树的**高度**至多为$2\log_2(n+1)$。

由引理可知，查找、插入和删除操作可在红黑树以$O(\log n)$时间内实现。

在实际应用中，红黑树的统计性能要好于AVL树，使用更多，但极端性能略差。

9.2.3 红黑树（续）

2. 红黑树的存储结构

根据插入和删除算法的特点，红黑树常用的存储结构是三叉链表，并设置一个全树共享的空节点。树中原来指向空节点的指针都指向这个共享空节点，而且根节点的parent指针也指向它。

```
typedef enum { RED = 0, BLACK = 1 } ColorType;   // 颜色类型
typedef struct RBNode {
   KeyType    key;      // 关键字
   ......               // 省略了应用相关的域，可根据需求添加
   ColorType color;     // 颜色
   RBNode    *lchild, *rchild, *parent;    // 三叉指针
} RBNode, *RBNodePtr;    // 红黑节点、红黑节点指针类型
 typedef struct {
    RBNode *root;    // 根指针
    RBNode *nil;     // 全树共享的空节点指针
 } *RBTree;          // 红黑树指针类型
```

```
RBTree MakeEmptyRBT() { // 构建空红黑树T
   RBTree T;
   if (!(T->nil = (RBNode*)malloc(sizeof(RBNode)))) exit(OVERFLOW);   // 结构记录
   T->nil->key = INFINITY;          // 空节点的key置为最大值
   T->nil->color = BLACK;           // 空节点的颜色置为黑色
   T->nil->parent = T->nil->lchild = T->nil->rchild = NULL;
   T->root = T->nil;                // 空树的根指针指向空节点（也称为nil节点）
   return T;                        // 返回构建的空红黑树
}
```

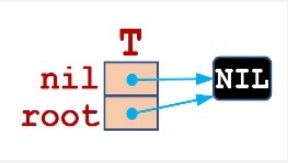
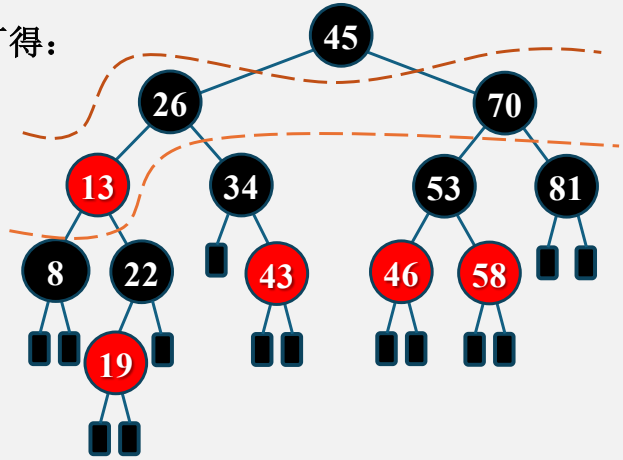


图9.16 空红黑树的存储结构

9.2.3 红黑树（续）

对空树依次插入 34, 53, 45, 26, 81, 70, 46, 8, 22, 58, 13, 43, 19，可得：



红黑树(red-black tree) 是满足以下性质的二叉查找树：

① 每个结点或是红的，或是黑的。

② 根结点是黑的。

③ 所有叶结点（nil）都是黑的。

④ 每个红结点的两个子结点必须是黑的。

⑤ 从任一结点到其每个后代叶结点的所有路径都包含相同个数的黑结点。

黑高度 ——从某个结点x出发(不包括结点x本身)到后代叶结点(包括叶结点)的路径上的黑结点个数，称为该结点x的黑高度，记为bh(x)；

根结点的黑高度就是该红黑树的黑高度；

叶结点的黑高度为0。

结点 8、19、22、34、43、46、53、58和81的黑高度均为1；

结点 13、26和70的黑高度为2；

根结点45的黑高度为3；

该红黑树的黑高度为3。

9.2.3 红黑树（续）

常用三叉链表作为红黑树的存储结构，便于插入和删除结点时，沿parent链向上进行红黑性质维护。

红黑树的根指针root和共享的nil叶结点被封装为一个结构体。

```
typedef struct RBNode {
  KeyType    key;     // 关键字
  ......              // 应用相关域
  ColorType color;  // 颜色
  struct RBNode *lchild,*rchild,*parent; //三叉指针
}RBNode, *RBNodePtr;   // 红黑结点，  红黑结点指针
typedef struct {
  RBNode *root;   // 根指针
  RBNode *nil;    // 共享的叶结点，初始构建空树时生成
}RBTree;   // 红黑树
```

为了便于处理红黑树算法操作的边界条件，采用一个共享哨兵来代表NIL。

哨兵T.nil结点的color域为BLACK，其他域可取任意值。

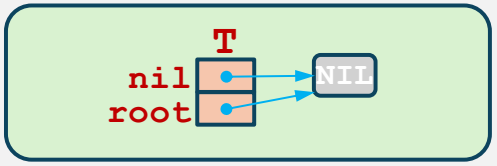
树中所有指向NIL的指针都被替换为指向哨兵T.nil结点的指针。

使用哨兵后，可将结点x的NIL孩子视为一个其父结点为x的普通结点。

引入nil结点，简化了算法的操作。

共享nil结点，又节省了存储空间。

红黑树的初始化是构造共享的nil结点和空树。




```
Status MakeEmptyRBT(RBTree &T) { // 构建空红黑树T
  if(!(T.nil = New_RBNode(-1, -99))) return OVERFLOW;
  T.nil->color = BLACK;    // 生成nil结点
  T.nil->parent = T.nil->lchild = T.nil->rchild = NULL;
  T.root = T.nil; // 空树的根指针指向nil结点
  return OK;
}
```

清華大學出版社 TSINGHUA UNIVERSITY PRESS

3. 红黑树的节点插入

插入节点的过程是：

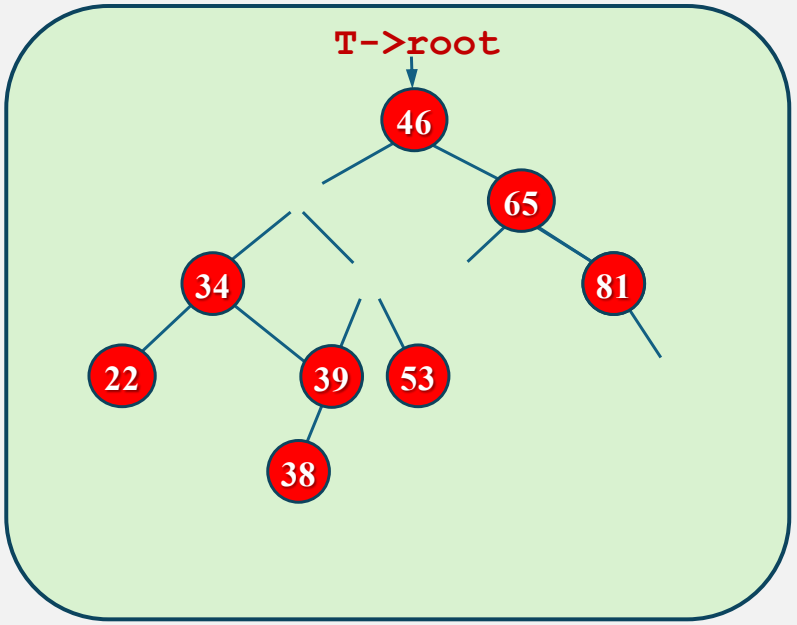
（1）从根节点向下搜索插入位置；

（2）用新节点替代某个已经存在的空节点，并令其拥有两个作为子节点的空节点；

（3）新节点置为红色，并确定其父节点的颜色，必要时沿parent链作自底向上的调整，维护红黑性质。

从空树开始，依次插入以下关键字的过程：

46 65 70 81 34 22 53 39 38



课外在AnyviewC可视跟踪观察算法过程中红黑树的形态变化

红黑树

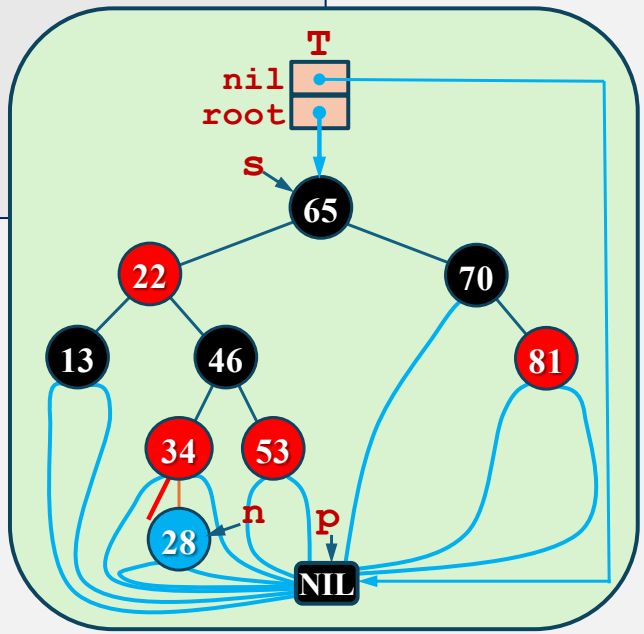
插入算法

```
Status InsertRBT(RBTree T, KeyType key, DataType data) {  // 算法9.14
  RBNode *n, *p = SearchRBT_Aux(T, key);                  // 插入算法
  if (p!=T->nil && p->key==key) return FALSE;  // 若已存在，则不插入
  if (!(n = (RBNode*)malloc(sizeof(RBNode)))) exit(OVERFLOW); // 分配
  n->key = key;  n->parent = p; // 在末梢插入
  n->lchild = n->rchild = T->nil;
  n->color = RED;                    // 标记红色
  if (p == T->nil) T->root = n;   // 为根结点
  else if (key < p->key) p->lchild = n; // 为左孩子
       else p->rchild = n;              // 为右孩子
  InsertFixup(T, n);    // 维护红黑性质
  return TRUE;
}
```

插入28，先要查找插入位置，

然后插入为p结点的左孩子

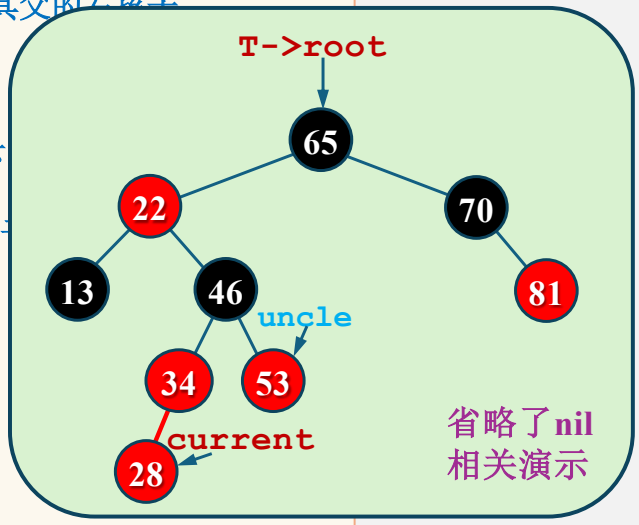
```
RBNodePtr SearchRBT_Aux(RBTree T, KeyType key) {
  // 算法9.15  插入和删除的辅助查找操作
  RBNodePtr p = T->nil,  s = T->root;
  while (s != T->nil) {
    p = s;
    if (key < s->key) s = s->lchild;        // 向左
    else if (key > s->key) s = s->rchild; // 向右
    else return p;  // p指向已存在的key结点
  }
  return p; // 未找到。key若插入，将作为p结点的孩子
}
```



红黑树

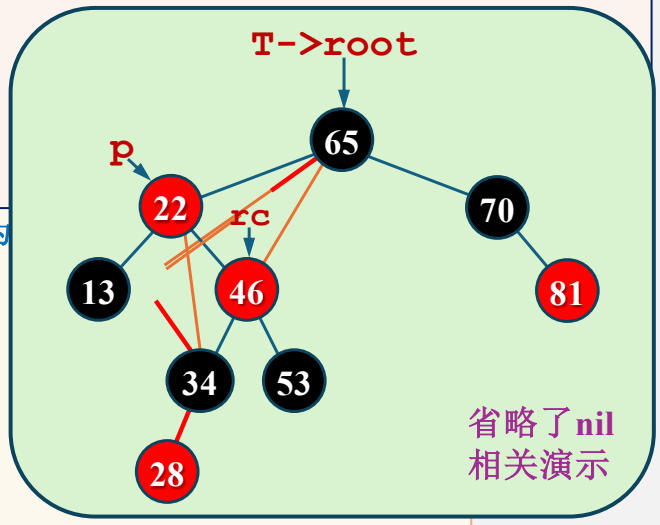
插入算法

```
void InsertFixup(RBTree T, RBNode *curr) { // 算法9.17 插入后再平衡
  RBNode *uncle;
  while(curr->parent->color == RED) {       // curr结点不是根且其父为红色
    if (curr->parent == curr->parent->parent->lchild) { // 如果其父是祖父的左孩子
      uncle = curr->parent->parent->rchild;    // 求其叔
      if (uncle->color == RED) {         // 如果其叔存在且是红色 (情况1)
/*情况1*/uncle->color = BLACK;                 // 其叔改为黑色
        curr->parent->color = BLACK;          // 其父改为黑色
        curr->parent->parent->color = RED; // 祖父置为红色
        curr = curr->parent->parent;     // curr指向祖父
      } else {                           // 否则其叔不存在，或者不是红色    (情况2)
        if (curr == curr->parent->rchild) { // 又如果curr结点是其父的右孩子
/*情况2*/  curr = curr->parent;           // curr指向其父
          L_Rotate_RBT(T, curr);         // 对其父为根的子树作左旋
        }
/*情况3*/curr->parent->color = BLACK;              // 当前父子关系下
        curr->parent->parent->color = RED;       // 置祖父为红色
        R_Rotate_RBT(T, curr->parent->parent); // 对以祖父为根的
      }
    } else {                             // 如果其父结点是祖父的右孩子
      /* 与左孩子的情形的对称操作 */
    }
  } //while 结束条件： curr结点是根结点，或者，其父结点是黑色
  T->root->color = BLACK;     // 置根结点为黑色
}
```



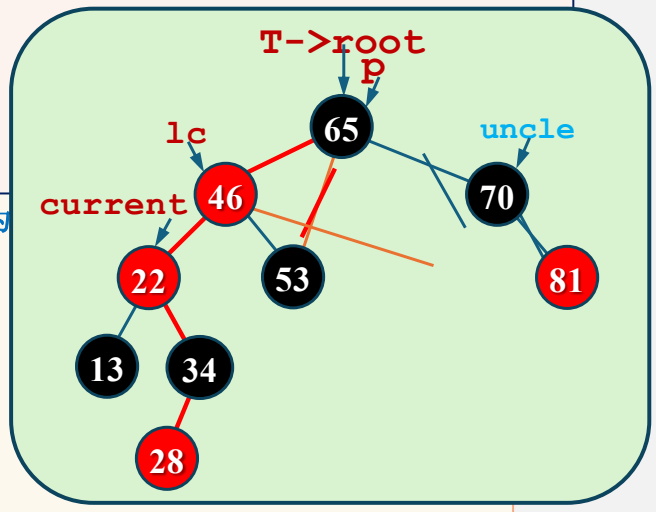
省略了nil
相关演示

```
Status InsertRBT(RBTree T, KeyType key, DataType data) {   // 插入算法
void InsertFixup(RBTree &T, RBNode *curr) { // 插入后再平衡
void L_Rotate_RBT(RBTree T, RBNode *p) { // 对红黑树T中p结点为根的子树进行左旋操作
  RBNode *rc = p->rchild; // rc指向p结点的右孩子
  p->rchild = rc->lchild; // 将rc结点的左子树转变为p结点的右子树
  if (rc->lchild != T->nil) rc->lchild->parent = p;
  rc->parent = p->parent
  if (p->parent == T->nil) T->root = rc; // 若p结点是原来的根，则rc结点成为新的根结点
  else if (p == p->parent->lchild)   // 否则若p结点是其双亲的左孩子
    p->parent->lchild = rc;          // 则rc结点是其新父结点的左孩子
  else p->parent->rchild = rc;       // 否则是右孩子
  rc->lchild = p;    // 将p结点置为rc结点的左孩子
  p->parent = rc;    // rc结点是p结点新的父结点
}
          R_Rotate_RBT(T, current->parent->parent); // 对以祖父为
        }
      } else {                          // 如果其父结点是祖父的右孩子
        /* 与左孩子的情形的对称操作 */
      }
    } //while 结束条件： curr结点是根结点，或者，其父结点是黑色
    T->root->color = BLACK;     // 置根结点为黑色
}
```



省略了nil
相关演示

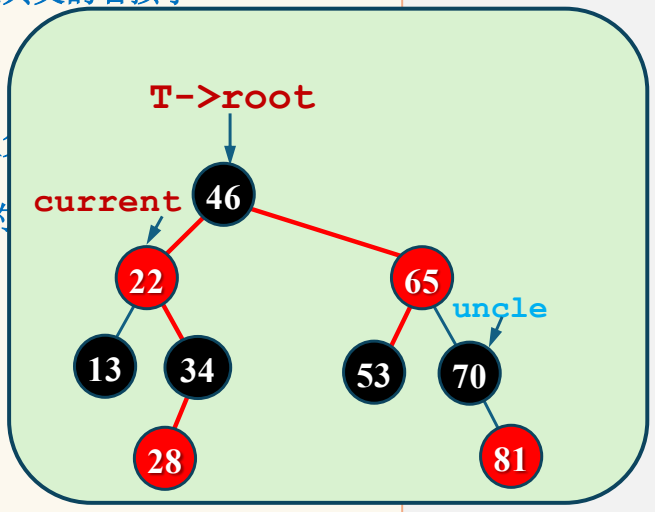
```
Status InsertRBT(RBTree T, KeyType key, DataType data) {   // 插入算法
 void InsertFixup(RBTree T, RBNode *curr) { // 插入后再平衡
void R_Rotate_RBT(RBTree T, RBNode *p) { // 对红黑树T中p结点为根的子树进行右旋操作
  RBNode *lc = p->lchild;       // lc指向p结点的左孩子
  p->lchild = lc->rchild;
  if (lc->rchild != T->nil) lc->rchild->parent = p; // 将lc结点的右子树转变为p结点的左子树
  lc->parent = p->parent;
  if (p->parent == T->nil) T->root = lc; // 若p结点是原来的根，则lc结点成为新的根结点
  else if (p == p->parent->rchild)       // 否则若p结点是其双亲的右孩子
         p->parent->rchild = lc;         // 则lc结点是其新父结点的右孩子
       else p->parent->lchild = lc;      // 否则是左孩子
  lc->rchild = p;    // 将p结点置为lc结点的右孩子
  p->parent = lc;    // lc结点是p结点新的父结点
}
          R_Rotate_RBT(T, current->parent->parent); // 对以祖父为
        }
      } else {                          // 如果其父结点是祖父的右孩子
         /* 与左孩子的情形的对称操作 */
      }
    } //while 结束条件： curr结点是根结点，或者，其父结点是黑色
   T->root->color = BLACK;     // 置根结点为黑色
}
```



红黑树

插入算法

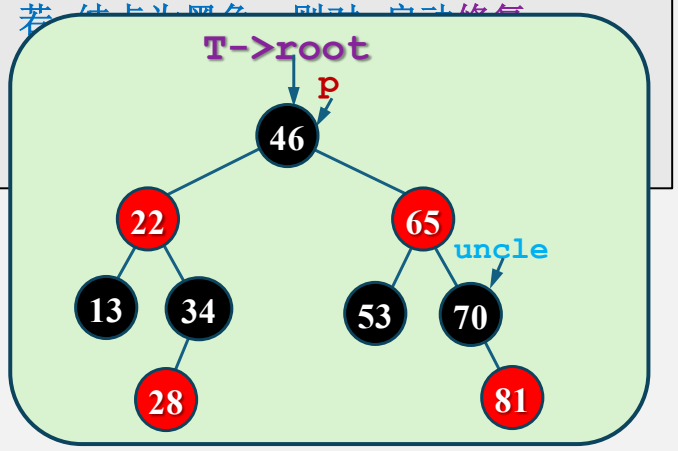
```
Status InsertRBT(RBTree T, KeyType key, DataType data) {   // 插入算法
void InsertFixup(RBTree T, RBNode *curr) { // 算法9.17  插入后再平衡
  RBNode *uncle;
  while(curr->parent->color == RED) { // p结点不是根且其父为红色
    if (curr->parent == curr->parent->parent->lchild) { // 如果其父是祖父的左孩子
      uncle = curr->parent->parent->rchild;    // 求其叔
      if (uncle->color == RED) {    // 如果其叔存在且是红色 (情况1)
        uncle->color = BLACK;                  // 其叔改为黑色
        curr->parent->color = BLACK;           // 其父改为黑色
        curr->parent-> parent->color = RED;    // 祖父置为红色
        curr = curr->parent->parent;           // curr指向祖父
      } else {                        // 否则其叔不存在，或者不是红色    (情况2)
        if (curr == curr->parent->rchild) { // 又如果curr结点是其父的右孩子
          curr = curr->parent;         // curr指向其父
          L_Rotate_RBT(T, curr);       // 对其父为根的子树作左旋
        }
        curr->parent->color = BLACK;          // 当前父子关系下，置
        curr->parent->parent->color = RED; // 置祖父为红色
        R_Rotate_RBT(T, curr->parent->parent); // 对以祖父为根的
      }
    } else {                          // 如果其父结点是祖父的右孩子
       /* 与左孩子的情形的对称操作 */
    }
  } //while 结束条件:  curr结点是根结点，或者，其父结点是黑色
  T->root->color = BLACK;     // 置根结点为黑色
}
```



红黑树

删除算法


```
Status DeleteRBT(RBTree T, KeyType key) {  // 算法9.18 在红黑树T中删除结点key
  RBNode *p = SearchRBT_Aux(T, key);
  if(p==T->nil || p->key!=key) return FALSE;  // 要删除的结点不存在，返回
  if(p->lchild==T->nil || p->rchild==T->nil) s = p; // p结点孩子不全则s指向它
  else s = SuccessorRBT(p); // 否则求p结点的按序后继（p结点的右子树的最小结点）
  if(s->lchild!=T->nil) c = s->lchild;  // c结点为s结点的左孩子
  else c = s->rchild;                   // c结点为s结点的右孩子（可能是NIL）
  c->parent = s->parent;   // 删除s结点
  if(s->parent==T->nil) T->root = c;
  else if(s==s->parent->lchild) s->parent->lchild = c;
       else s->parent->rchild = c;
  if(s!=p){ p->key = s->key;  ......  }  // 用s结点值顶替被删结点值
  if(s->color==BLACK) DeleteFixup(T, c); // 若
  free(s); // 释放s结点
  return TRUE;
}
```



清華大學出版社

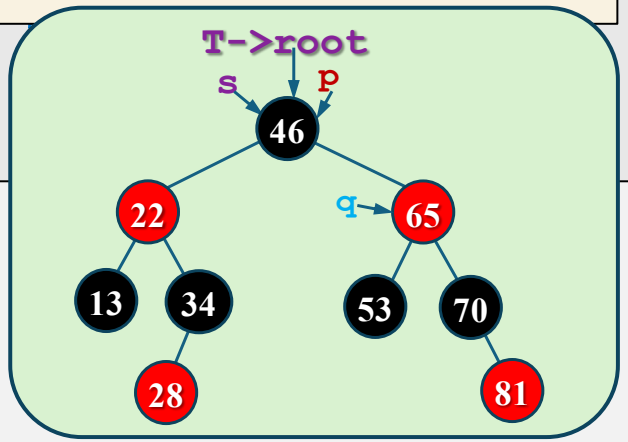
红黑树

删除算法

```
Status DeleteRBT(RBTree T, KeyType key) {  // 在红黑树T中删除结点key
  RBNode *p = SearchRBT_Aux(T, key);
  if(p==T->nil || p->key!=key) return FALSE;  // 要删除的结点不存在，返回
  if(p->lchild==T->nil || p->rchild==T->nil) s = p; // p结点孩子不全则s指向它
```

```
RBNode *SuccessorRBT(RBTree T, RBNode *s) { // 求s结点的按序后继结点
  if(s->rchild!=T->nil) return MinimumRBT(T, s->rchild); // 在非空右子树找到
  p = s->parent;  // 否则p指向s结点的父结点
  while(p!=T->nil && p->rchild==s) { // p向上走，直到为T->nil,则s结点是做大的，无后继
    s = p;  p = p->parent;          // 或者p结点的前一个结点是其左孩子，则找到s结点的后继
  }
  return p; // 返回指向后继的指针（若s结点已是最大，则返回T->nil）
}
```

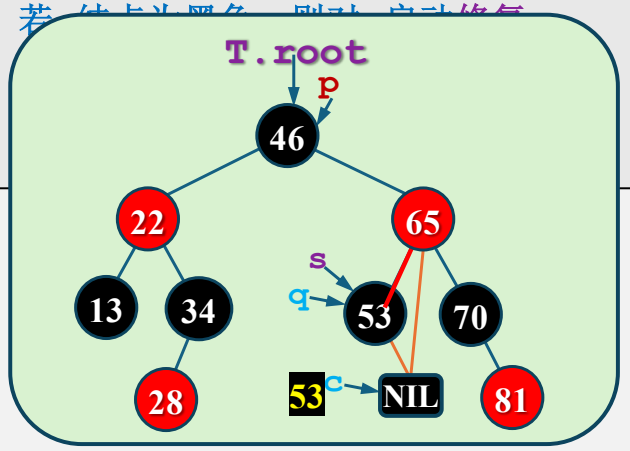
```
RBNode *MinimumRBT(RBTree T, RBNode *q) {
  // 求q子树的最小值结点
  while(q->lchild!=T->nil) q = q->lchild; // 向左到尽头
  return q;
}
```



红黑树

删除算法

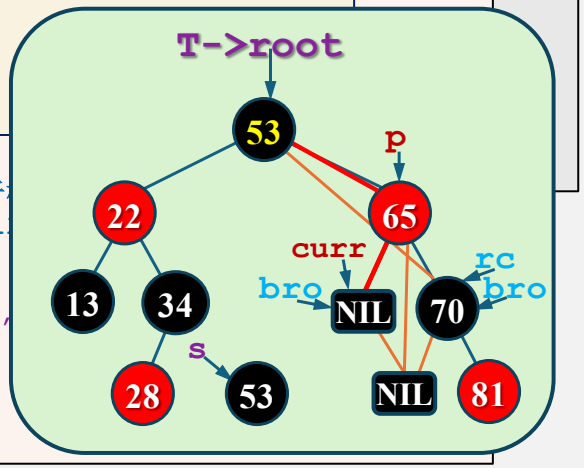
```
Status DeleteRBT(RBTree T, KeyType key) {  //算法9.18  在红黑树T中删除结点key
  RBNode *p = SearchRBT_Aux(T, key);
  if(p==T->nil || p->key!=key) return FALSE;  // 要删除的结点不存在，返回
  if(p->lchild==T->nil || p->rchild==T->nil) s = p; // p结点孩子不全则s指向它
  else s = SuccessorRBT(p); // 否则求p结点的按序后继（p结点的右子树的最小结点）
  if(s->lchild!=T->nil) c = s->lchild;  // c结点为s结点的左孩子
  else c = s->rchild;                   // c结点为s结点的右孩子（可能是NIL）
  c->parent = s->parent;   // 删除s结点
  if(s->parent==T->nil) T->root = c;
  else if(s==s->parent->lchild) s->parent->lchild = c;
       else s->parent->rchild = c;
  if(s!=p){ p->key = s->key;  ......  }  // 用s结点值顶替被删结点值
  if(s->color==BLACK) DeleteFixup(T, c); // 若
  free(s); // 释放s结点
  return TRUE;
}
```



课外在AnyviewC可视跟踪观察算法过程中
红黑树的形态变化

```
void L_Rotate_RBT(RBTree T, RBNode *p) { // 对红黑树T中p结点为根的子树进行左旋操作
  rc = p->rchild; // rc指向p结点的右孩子
  p->rchild = rc->lchild; // 将rc结点的左子树转变为p结点的右子树
  if (rc->lchild != T->nil) rc->lchild->parent = p;
  rc->parent = p->parent;
  if (p->parent == T->nil) T->root = rc; // 若p结点是原来的根，则rc结点成为新的根结点
  else if (p == p->parent->lchild)  // 否则若p结点是其双亲的左孩子
    p->parent->lchild = rc;         // 则rc结点是其新父结点的左孩子
  else p->parent->rchild = rc;      // 否则是右孩子
  rc->lchild = p; // 将p结点置为rc结点的左孩子
  p->parent = rc; // rc结点是p结点新的父结点
}
```

```
void DeleteFixup(RBTree T, RBNode *curr) {  // 算法9.19 对红黑树T中curr结点及相应部分进行修复
  while(curr!=T->root && curr->color==BLACK) // 当curr结点不是根而且是黑色（NIL总是黑色）
    if(curr == curr->parent->lchild) {  // 若是左孩子
      bro->rchild->color = BLACK;        // 置bro结点的右孩子为黑色
      L_Rotate_RBT(T, curr->parent);     // 对以curr结点的父结点为根的子
      curr = T->root;                    // curr指向根结点(强制退出whi
    }
  } else {   // 否则是右孩子
    // 此部分是与左孩子情形的对称操作（"lchild"和"right"互换、"左旋"和"右旋"
  }
  curr->color = BLACK;  // 置curr结点为黑色
}
```

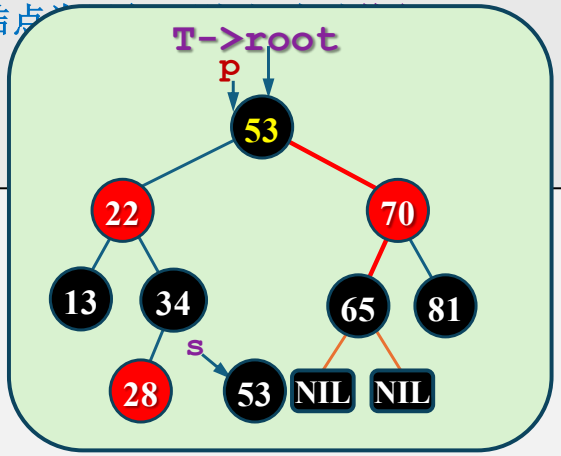


清華大學出版社

红黑树

删除算法

```
Status DeleteRBT(RBTree T, KeyType key) {  // 算法9.18 在红黑树T中删除结点key
  RBNode *p = SearchRBT_Aux(T, key);
  if(p==T->nil || p->key!=key) return FALSE;  // 要删除的结点不存在，返回
  if(p->lchild==T->nil || p->rchild==T->nil) s = p; // p结点孩子不全则s指向它
  else s = SuccessorRBT(p); // 否则求p结点的按序后继（p结点的右子树的最小结点）
  if(s->lchild!=T->nil) c = s->lchild;  // c结点为s结点的左孩子
  else c = s->rchild;                   // c结点为s结点的右孩子（可能是NIL）
  c->parent = s->parent;   // 删除s结点
  if(s->parent==T->nil) T->root = c;
  else if(s==s->parent->lchild) s->parent->lchild = c;
       else s->parent->rchild = c;
  if(s!=p){ p->key = s->key;  ......  }  // 用s结点值顶替被删结点值
  if(s->color==BLACK) DeleteFixup(T, c); // 若s结点
  free(s); // 释放s结点
  return TRUE;
}
```



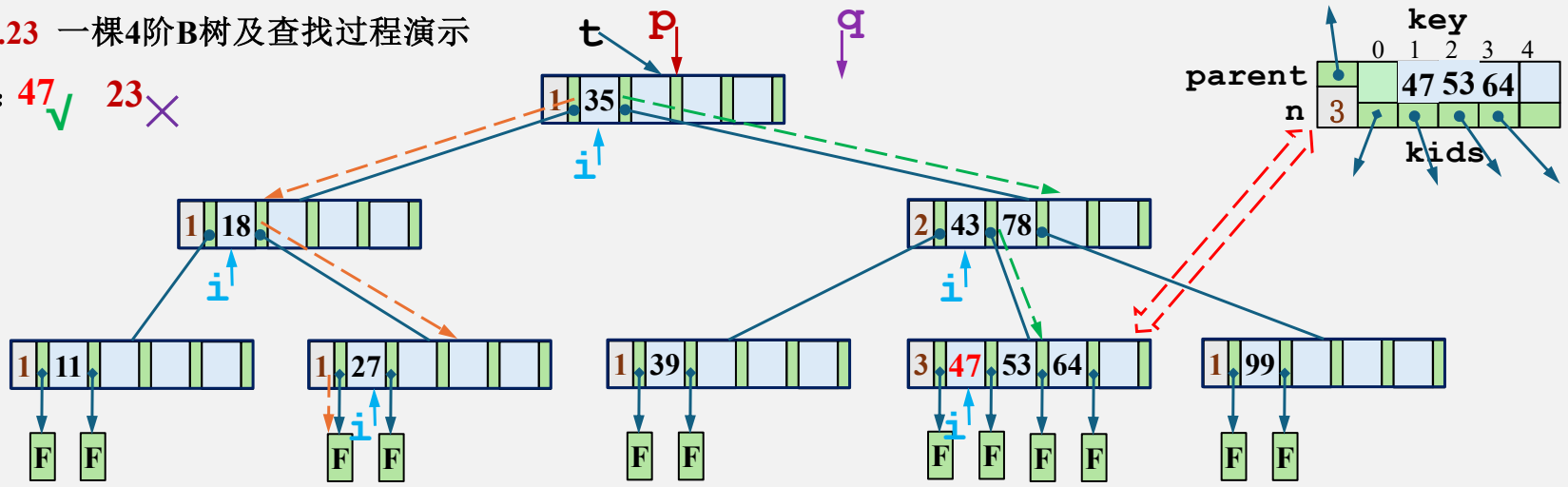
9.2.4 B树和B+树

B树的查找

```
Result Search(BTree T, KeyType key) { // 算法9.20
  // 在m阶B树T上查找关键字K，返回结果(pt,i,tag)
  p=T;  q=NULL;  found=FALSE;  i=0;    // 初始化
  while (p && !found) {
    i=Find(p, key);  // 在p->keys[1..p->n]中查找i,
                     // 使得: p->keys[i]<=K<p->keys[i+1]
    if (i>0 && p->keys[i]==key) found=TRUE; // 找到待查关键字
    else { q=p;  p=p->kids[i]; if(p)p->pci=i; } // 下一层继续查找
  }
  if (found) return(p,i,1);   // 查找成功，返回K的位置
  else return(q,i,0);         // 查找不成功，返回K的插入位置
}
```

图9.23 一棵4阶B树及查找过程演示

查找：47√ 23×



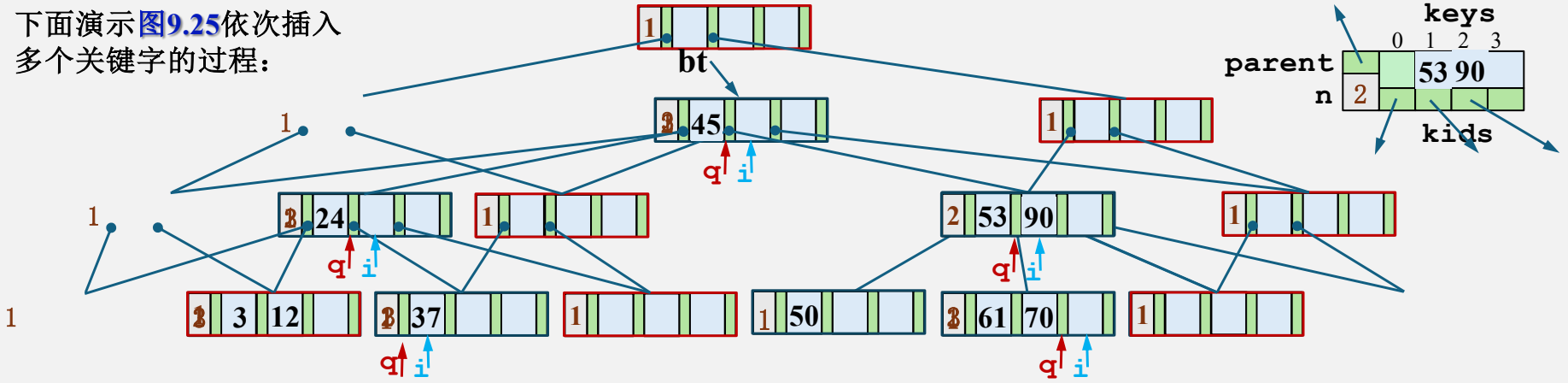
B树的插入

通过算法9.20确定key的插入位置后，可调用算法9.21在m阶B树T上*q结点的keys[i]与keys[i+1]之间插入关键字key。

若引起结点过大，则沿双亲链进行必要的结点分裂调整，使T仍是m阶B树。

```
BTree InsertBTree(BTree T, KeyType key, BTree q, int i) {  // 算法9.21
  if (!T) { T = NewRoot(NULL, key, NULL);  return T; }  // T空则生成含key的根
  x=key;  sp=NULL;  finished=needNewRoot=FALSE;  // 初始化
  while (!needNewRoot && !finished) {
    Add(q, i, key, sp);      // 将x和ap分别插入到q->key[i+1]和q->ptr[i+1]
    if (q->n < m) finished = TRUE;  // 插入完成
    else {  // 分裂结点*q
      s = (m+1)/2;  sp = split(q, s);  x = q->key[s];
      if(q->parent) { i=q->pci;  q=q->parent; } // 在双亲结点中x插入位置为i之后
      else needNewRoot=TRUE;
    }
  }
  if(needNewRoot)T=NewRoot(q,recs,sp); // T是空树，或者根结点已分裂，生成新根结点*T
  return T;
}
```

下面演示图9.25依次插入多个关键字的过程：



插入：87

图9.25 3阶B树的插入过程演示 (a) (b) (c) (d) (e) (f) (g) (h) (i) (j)

B树的删除

清華大學出版社 TSINGHUA UNIVERSITY PRESS

位于子树的最左下的底层结点内

B树上删除q结点的关键字K_i：

(1) 若所删的是非底层结点的K_i，则可用指针A_i所指子树中最小关键字Y替代K_i，然后在相应结点删去Y；例如删除：45

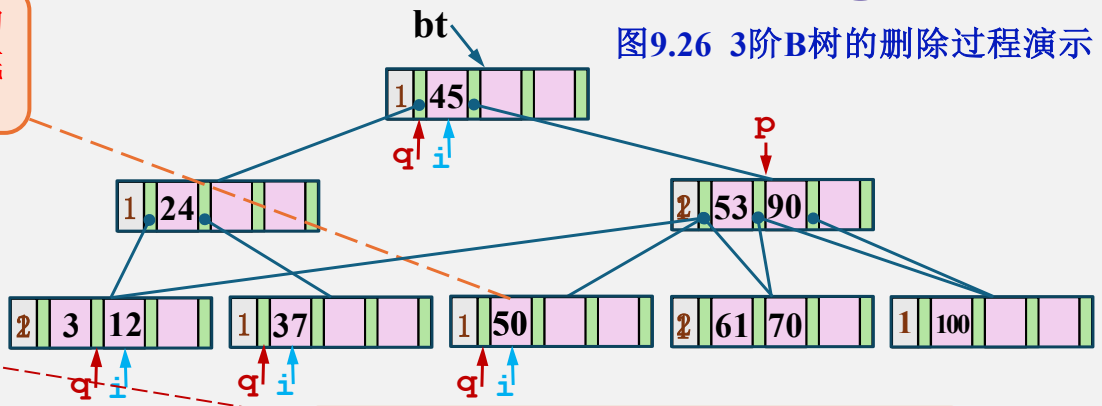


图9.26 3阶B树的删除过程演示

留待讲解删除底层结点的关键字时演示

(2) 若所删的是底层结点的K_i，需分三种情形：

① K_i所在结点$n \geq \lceil m/2 \rceil$，只需从该结点删除K_i和指针A_i，其余不变；例如删除：12

② K_i所在结点$n = \lceil m/2 \rceil - 1$，其相邻的右（左）兄弟结点的$n > \lceil m/2 \rceil - 1$，则需将其兄弟结点中的最小（最大）的关键字上移到父结点保序位置，并将父结点中小于（大于）且紧靠刚上移的关键字的关键字下移到被删关键字K_i的所在结点的位置；例如删除：50

③ K_i所在结点和其相邻的兄弟结点均$n = \lceil m/2 \rceil - 1$，若有右兄弟（假设被父结点的指针$A_j$所指），则删去$K_i$和$A_i$后，其所在结点中剩余的关键字和指针，加上父结点中的指针A_j一起，合并到A_j所指的兄弟结点中（若没有右兄弟，则合并到左兄弟结点中）。例如删除：53

如果因此使父结点的$n < \lceil m/2 \rceil - 1$，则依次类推作相应处理。例如删除：37

可在AnyviewC可视跟踪观察算法过程中B树的形态变化

B+树简述

在第12章的12.3.3节，将详细讨论B⁺树的实现

B⁺树是应文件系统所需而对B树的改进变形。m阶B⁺树和m阶B树的主要差异如下。

(1) 含n棵子树的节点中含n个关键字。

(2) 所有叶节点中包含了全部关键字的信息及指向含这些关键字记录的指针，且叶节点本身依关键字的大小从小到大顺序链接。

(3) 所有的内部节点可以看作索引部分，节点中仅含其子树(根节点)中的最大(或最小)关键字。

通常B⁺树有两个头指针：一个指向根节点，另一个指向关键字最小的叶节点。因此B⁺树的查找方式有两种：

一种是从最小关键字起顺序查找；

另一种是从根节点开始，进行随机查找。

如图9.27所示，其中sqt代表从最小关键字起顺序查找的指针，最下层的叶节点都有指向含这些关键字记录的指针。

在B⁺树上进行随机查找、插入和删除的过程基本上与B树类似。只是在查找时，若内部节点上的关键字等于给定值，并不终止，而是继续向下查找直到叶节点。因此，在B⁺树中，不管查找成功与否，每次查找都是走了一条从根到叶节点的路径。

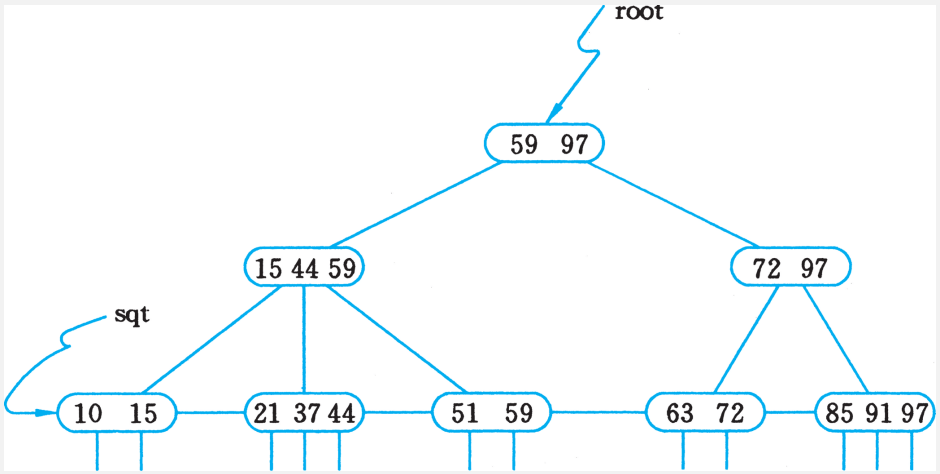


图9.27 3阶B+树示例

9.2.5 跳跃表

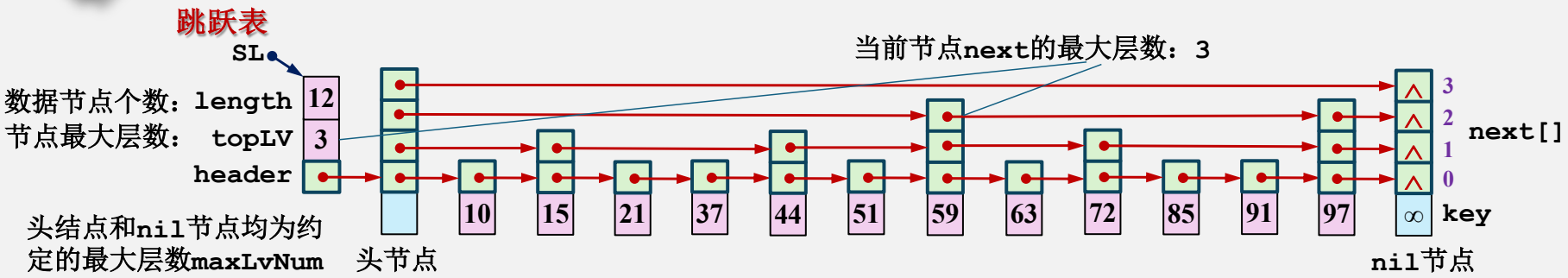


图9.27的3阶B⁺树也可以表示为图9.28的跳跃表。做法是：

（1）B⁺树所有节点拆分为单关键字节点，且相同关键字共享一个节点，按个数设置指针域；

（2）像底层节点那样，每一层节点都横向构成一个有序链表；

（3）各层的横向链表的头指针构成一个头节点；

（4）增设一个由全局指针变量nil指示的尾节点，其关键字为最大值，各层指针为空；

（5）头节点和nil节点的指针层数均为约定的最大层数。

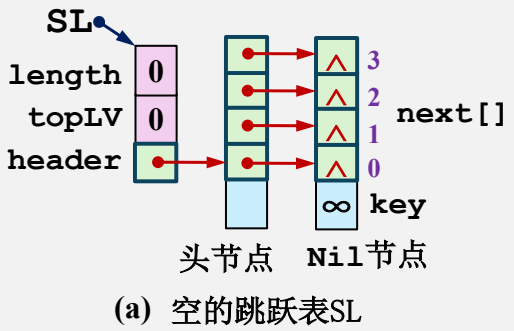
（6）跳跃表是一个含三个域的结构体，length值是底层链表的长度（元素节点数），topLV是当前元素节点的最大层数，header指向头节点。

说明：跳跃表是层次结构，其“结点”随树结构也称为“节点”。

1. 跳跃表及其查找

```
#define  minKey    -RAND_MAX   // 最小关键字
#define  maxKey     RAND_MAX   // 最大关键字
#define  maxLVnum    4         // 允许最大层数（可根据实际需求定义）
typedef int  KeyType;          // 关键字类型（可根据实际需求定义）
typedef char ValueType;        // 值类型（可根据实际需求定义）
typedef struct SLNode {
   KeyType     key;            // 关键字
   ValueType   value;          // 数据项
   int         lvNum;          // 结点层数（指针数组大小）
   SLNode    **next;           // 动态分配的指针数组
} SLNode;                      // 跳跃表结点类型
SLNode    *nil;                // 跳跃表的共享尾结点的指针
typedef struct {
   int      length;           // 表长（表中数据项数）
   int      topLV;            // 当前跳表的层数
   SLNode *header;            // 头结点指针
} *SkipList;                   // 跳跃表类型
```

图9.29(a)是一个空跳跃表，头节点的各层指针均指向nil节点。



(a) 空的跳跃表SL

1. 跳跃表及其查找（续）

```
ValueType SearchSL(SkipList sl, KeyType key) {   // 算法 9.23
    // 在跳跃表sl查找关键字为key的元素，并返回其值
    SLNode *p, *q;    int lv;
    for (q=sl->header, lv=sl->topLV-1; lv>=0; lv--) {      // 自顶向下逐层查找
        p = q->next[lv];
        while (key > p->key) { q = p;  p = p->next[lv]; }   // 在lv层有序链表查找
        if (key == p->key) return p->value;    // 返回找到的元素值
    }
    return nullValue;   // 未找到，返回空值
}
```

查找85　ValueType v = SearchSL(SL, 85);　key = 85

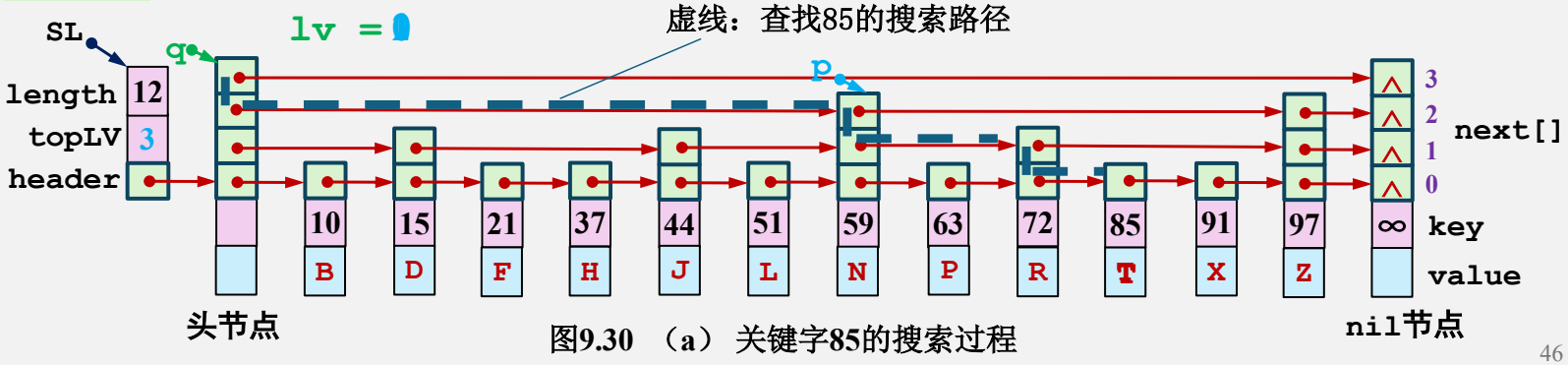


图9.30 （a） 关键字85的搜索过程

2. 跳跃表的插入和删除

在跳跃表中插入一个新结点时，首要任务是确定该结点的指针层数。所有结点至少有一个指针。按照图9.29(b)所示，如果每2个结点中就有一个结点的指针至少是两个，则我们可以在第2层上每次跳过2个结点，不断地迭代，直到每2^j个结点中，就有一个结点至少有j+1个指针。

为使结点具有这种性质，可使用一个以概率$1/t^i$返回i+1的值的函数randTopLV进行新结点的层数随机化。用返回的随机层数i，创建一个具有i层指针的新结点。

在图9.30(a)所示的跳跃表SL插入88，结果如图9.30(b)所示。如何实现在跳跃表的插入具体操作呢？和二叉排序树类似，要先进行查找，确定表中不存在要插入的关键字，才能实施插入。但是，二叉排序树只有一个插入点，而跳跃表要在多层链表插入同一个结点，而且查到0层链表才能确认该关键字不存在，此时才能新建结点并逐层插入。算法9.24采用指针数组r[0.. maxLVnum-1]，在各层有序链表的查找中保存了插入位置的前驱指针。在0层链表确认关键字不存在后，新建具有随机层数的结点，并按其层数，用数组r辅助实现相应各层的插入。

```
int randTopLV() {   // 按既定概率获取新结点的层数
   r = rand();      // 产生随机数
   for (int i=1, j=2; i<maxLVnum; i++, j+=j)     // j=2,4,8,16,...
      if (r > RAND_MAX/j) break;  //按1/2, 1/4, 1/8, 1/16...概率定1,2,3,4...层数
   return i;        // 返回新结点的层数
}
```

跳跃表的插入

```
Status InsertSL(SkipList sl, KeyType key, ValueType value) {   // 算法 9.24 非递归插入
  SLNode *p, *q, *x;    int lv;     SLNode *r[maxLVnum];
  q = sl->header;              // q指向头结点，用指针数组r[0..maxLVnum-1]辅助插入
  for (lv = maxLVnum-1; lv>=0; lv--) r[lv] = q;   // r的各层指针均指向头结点
  for (lv = sl->topLV-1; lv >= 0; lv--) {          // 自上而下逐层保序插入
    p = q->next[lv];                          // p指向q结点在lv层的后继结点
    while (key > p->key) { q = p;    p = p->next[lv]; }   // 查找插入位置
    if (key == p->key) return FALSE;          // 若已存在，则不重复插入
    r[lv] = q;                                // r[lv]保存指向lv层插入位置前驱结点的指针
  }      // for循环结束时，r各指针分别指向各层链表插入位置的前驱结点
```

查找插入位置

比算法9.23增加了红色两行，辅助插入定位

插入88　Status tf = InsertSL(SL, 88, 'V');　key = 85　value = 'V'

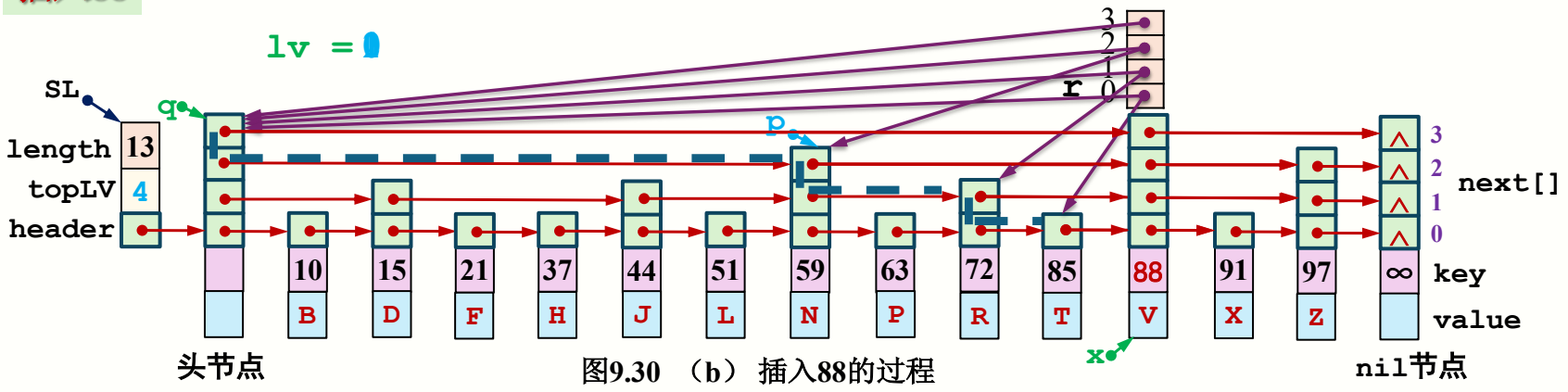
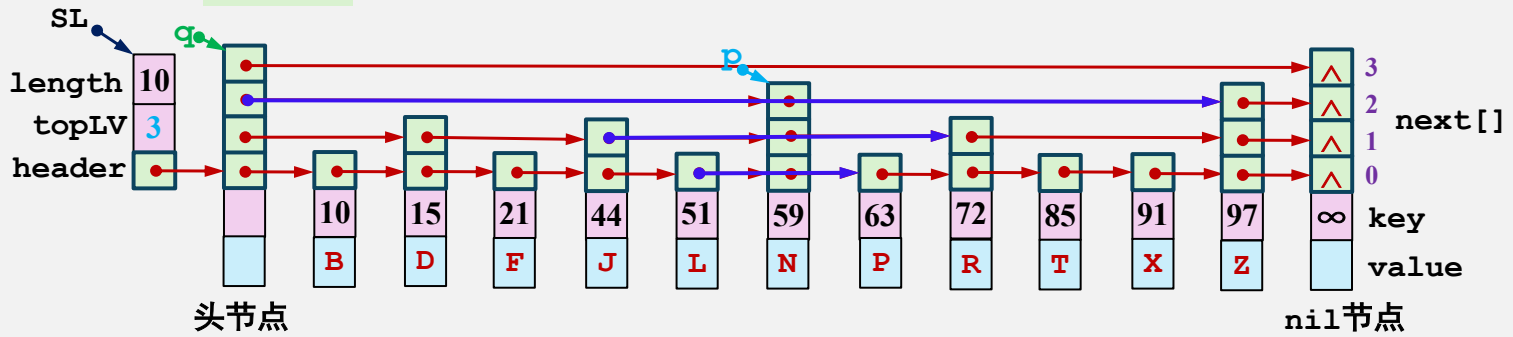


图9.30 （b） 插入88的过程

跳跃表的删除

删除59　Status tf = DeleteSL(SL, 59);　key = 59



在跳跃表删除结点相对简单，算法9.25基于查找算法的架构，自上而下直接实现了目标结点在各层链表的删除。

在0层链表删除后，释放被删除结点，表的长度减1，成功返回。若算法执行到末尾，则表明目标结点不存在，无结点被删除。

```
Status DeleteSL(SkipList sl, KeyType key) {  // 算法 9.25  非递归删除
  SLNode *p, *q = sl->header;    // q指向头结点
  for (int lv = sl->topLV-1; lv >= 0; lv--) {  // 自上而下逐层搜索
    p = q->next[lv];             // p指向q结点在level层的下一个结点
    while (key > p->key) { q = p;  p = p->next[lv]; }  // 在level层搜索
    if (key == p->key) {                    // 若p结点是要删结点
      q->next[lv] = p->next[lv];            // 则在本层链表删除
      if (lv == 0) {                        // 若已在0层链表删除
        free(p->next);    free(p);          // 则释放被删结点空间
        sl->length--;     return TRUE;      // 成功返回
      }
    }
  }
  return FALSE;   //
}
```

在自上而下各层查找过程中，遇目标节点，则在该层删除

平均情况下，这种跳跃表的**查找、插入和删除**操作大约需要$(2\log_2 n)/2$次比较，是$O(\log_2 n)$的。

清華大學出版社
TSINGHUA UNIVERSITY PRESS

9.2.6 键树

双链键树的构建、回收、查找和插入算法的学习，
可在AnyviewC进行可视交互跟踪观察树形的动态变化

1. 双链键树——孩子兄弟链表表示的键树

采用孩子兄弟链表存储结构的键树称为**双链键树**。每个分支节点含3个域：symbol域存储关键字的一个字符，first指针域指向第一棵子树根，next指针域指向右兄弟。叶子节点除了symbol和next域，还有指向关键字记录的指针域inforptr，但没有first指针域。

例如，图9.31所示的键树的双链树如图9.32(a)所示（只画出第一棵子树，省略了其余部分）。

```
typedef char *KeyType;     // 关键字类型是C串
typedef char *InfoType;    // 关键字对应的信息类型
typedef struct{
   KeyType   key;          // 关键字
   InfoType  info;         // 记录的其它部分
} Record;                  // 记录类型
typedef enum { LEAF, BRANCH } NodeKind; // 节点种类:{叶子,分支}
typedef struct DLTNode {
   char       symbol;      // 关键字中的一个字符
   DLTNode   *next;        // 指向兄弟节点的指针
   NodeKind   kind;        // 节点种类
   union {
      Record  *rcd;        // 叶子节点的记录指针
      DLTNode *first;      // 分支节点的孩子链指针
   };
} DLTNode, *DLTree;        // 节点类型，双链键树指针类型
```

9.3 散列表

9.3.1 散列和散列表定义

理想的情况是不经过任何比较，一次存取便能得到所查元素。这就必须在元素的存储位置和它的关键字之间建立一个确定的对应关系f，使每个关键字和结构中一个唯一的存储位置相对应。于是在查找时，只要根据这个对应关系f便可找到给定值K的像$f(K)$。若结构中存在关键字等于K的元素，就必定在存储位置$f(K)$上，则不需要进行比较便可直接取得所查元素。这个对应关系f称为**散列**（Hash）**函数**或**哈希函数**，按这一思想建立的查找表称为**散列表**或**哈希表**。散列表设有一个称为**向量**的长度为size的动态一维数组，其下标范围0..size对应散列函数的值域。散列表具有与动态查找表类似的基本操作。

1. 加密散列函数

加密散列函数将变长的位字符串输入固定地映射为定长的位字符串输出。如前所述，散列碰撞是不可避免的，但是一个安全的散列函数需要具备抗冲突性，应该很难发生冲突。这类函数需要数学证明无冲突。加密散列函数还必须具备抗原像性（单向特性），即不能进行逆运算，不能从散列结果逆向还原输入。

加密散列函数在密码学中非常重要，并且广泛应用于数字签名、模式验证、区块链和消息完整性等许多实际场景。

2. 非加密散列函数

非加密函数不需要安全性和很强是抗冲突性，但必须能快速计算并且保证较低的散列冲突概率，因而允许以合理的错误概率快速对大量数据进行散列。用于散列表的散列函数是非加密的，通常比加密的散列函数的计算速度快5至数十倍。

常用的散列函数构造方法

构造散列函数时，一般都要对关键字进行计算。为了尽量避免不同的关键字产生相同的散列函数值，应使关键字的所有组成成分都能对散列函数的计算起作用。

（1）直接定址法

取关键字或关键字的某个线性函数值为散列地址，即

$$H(key) = key \quad 或 \quad H(key) = a \times key + b$$

其中a和b为常数（这种散列函数叫做自身函数）。

（2）数字分析法

假设关键字是以r为基的数（比如以10为基的十进制数），且散列表中可能出现的关键字事先已知，可根据散列表的长度取关键字的若干数位组成散列地址。

（3）平方取中法

取关键字平方值的中间若干位作为散列地址。一个数的平方值的中间几位与数的每一位都相关，由随机分布的关键字得到的这种散列地址也是随机的。求平方后取的位数由散列表的长度决定。

（4）折叠法

将关键字分割成位数相同的几部分（最后一部分的位数可以不同），然后取这几部分的叠加和（舍去最高位进位）作为散列地址，这种方法称为**折叠法**（folding）。当关键字位数很多，而且每位数字分布大致均匀时，可以采用此法求散列地址。

（5）随机数法

选择一个随机函数，取关键字的随机函数值为其散列地址，即

$$H(key) = \text{random}(key)$$

其中random为随机函数。当关键字长度不等时，通常采用此法构造散列函数较恰当。

（6）除留余数法

被散列表表长m除后所得余数作为散列地址，称为模散列函数。前5种方法构造的散列函数所求的散列值都必须对表长m取模才能成为散列表的合法地址。

$$H(key) = key \text{ MOD } m$$

关键字和表长对构造散列函数的影响

下面分别讨论关键字*key*和表长*m*对构造散列函数的影响。

先看关键字。严格地说，对于每一种可能用到的关键字的类型，需要选择相应的运算。出于效率的考虑，通常希望避免显式类型转换，将机器码中关键字的二进制表示看作进行算术计算的数。

浮点数关键字。最简单的情况可能是将固定范围内已知的浮点数作为关键字。例如，关键字是0到1之间的实数，可以把每一个数乘以表长*m*，然后取最接近的整数，得到0到*m*-1之间的整数地址。如果关键字*k*都落在区间[*s*..*t*]，则可以通过*k*减*s*再除以(*t*-*s*)，可得到0和1之间的实数，然后乘以*m*就得到一个表中地址。

整数关键字。*n*位整数关键字最常用的散列方法是选择表长*m*为素数，对于任何整数关键字*k*，计算*k*除以*m*的余数，即计算$h(k) = k \bmod m$。$h(k)$就是**模**散列函数。这种运算易于实现（在C语言中为*k*%*m*），而且可以高效地将整型关键字均匀打散。

字符串关键字。在很多符号表和大数据应用中，关键字不是整数，而是较长的字母序列（或字节序列）。如何计算字符串averylongkey的散列值呢？在7位ASCII码中，该关键字对应于如下84位整数：

$$\begin{aligned}&97\cdot128^{11} + 118\cdot128^{10} + 101\cdot128^{9} + 114\cdot128^{8} + 121\cdot128^{7}\\&+ 108\cdot128^{6} + 111\cdot128^{5} + 110\cdot128^{4} + 103\cdot128^{3}\\&+ 107\cdot128^{2} + 101\cdot128^{1} + 121\cdot128^{0}\end{aligned}$$

该数太大，无法在大多数计算机中用普通算术函数表示。但在实际应用中，还可能需处理更长位数的关键字。

可以通过分段转化关键字来计算长关键字的模散列函数值，就是利用mod函数的算术性质并使用霍纳（Horner）算法。这种算法是基于关键字的另一种表示来求散列值的方法。例如，上述式子可写成如下表达式（10层括号）：

$$\begin{aligned}&(((((((((97\cdot128 + 118)\cdot128 + 101)\cdot128 + 114)\cdot128\\&+ 121)\cdot128 + 108)\cdot128 + 111)\cdot128 + 110)\cdot128\\&+ 103)\cdot128 + 107)\cdot128 + 101)\cdot128 + 121\end{aligned}$$

我们不是要计算该数，而是它除以*m*后的余数。可在运算每一步都去掉*m*的倍数，即在每次乘加运算之后只保存模*m*的值。这相当于计算了长整数之后再对*m*求模。这就是直接使用算术方法计算长字符串的模散列函数`hash`，可用素数127代替128做基数。

清華大學出版社
TSINGHUA UNIVERSITY PRESS

长字符串的模散列函数hash

```
int hash(char *k, int M) {
    int h, a;
    for (h=0, a=127;  *k!='\0';  k++)
        h = (a*h + *k) % M;
    return h;
}
```

采用素数作为基数，而不是与字符串的ASCII表示对应的整数定义中要求的2的幂作为基数。但是，由这个hash产生的散列值对表长为127的倍数的表性能可能不佳。改进的方法是在计算中使用随机系数，并且对关键字中每位的系数采用不同的随机值。这种算法称为通用散列算法。

一个理想的通用散列函数使得表中两个不同关键字产生冲突的机率为表长度m的倒数，即$1/m$。可以证明，采用一个不同随机值，而非固定值的序列作为hash中的系数，可使模散列函数成为一种通用散列函数。这可以通过使用一个简单的伪随机序列对应一个关键字的字符位置的系数，将hash改进为字符串通用模散列函数hashU，如算法9.30所示。

```
int hashU(char *v, int m) {   // 算法 9.30
    int h, a = 31415, b =27183;
    for(h=0;  *v!='\0';  v++, a=a*b%(m-1))    // 每次计算下一个伪随机系数a
        h = (a*h + *v) % m;
    return h;
}
```

hashU的计算与hash相同。但使用伪随机系数代替固定基数，以便两个不等关键字的冲突概率近似为1/m。

再来看模散列函数的模m。构造散列表时，要注意对表的长度m的选择。若m选不好，容易产生同义词。

例9-9　当m=21（=3×7）时，下列含因子7的值对21取模所得到的散列地址均为7的倍数：

关键字	28	35	63	77	105
散列地址	7	14	0	14	0

此例说明，若m含有质因子pf，则所有含有pf因子的关键字的散列地址均为pf的倍数。一般情况下，可以选取表长m为素数或者不含小于20的质因子的合数。

在实际应用中，应视不同情况采用不同的散列函数。考虑的主要因素有：

(1) 计算散列函数所需时间（包括硬件指令的因素）；

(2) 关键字类型；

(3) 关键字长度；

(4) 散列表大小；

(5) 关键字的分布情况；

(6) 数据元素的查找频率。



9.3.3 冲突处理方法和散列表构造

假设散列表的地址区间为[0, m-1]，冲突是指由关键字得到的散列地址为j($0 \le j \le m$-1)的位置上已存有另一关键字不同的元素。“处理冲突”就是为该关键字的元素找到另一个“空单元”的散列地址。在处理冲突的过程中，可能得到一个地址序列 H_i（i=1,2,…,k, $H_i \in$[0,m-1]）。即在处理散列地址的冲突时，若得到的下一个散列地址H_i仍然发生冲突，则再求下一个地址H_{i+1}。依次类推，直到H_k不发生冲突为止，H_k即为元素在表中的地址。

处理冲突的常用方法有下列几种，散列表的相应存储结构也有所不同。

1. 拉链法

拉链法也称为链地址法，将所有关键字为同义词的元素存储在同一个双向链表中（该链表也称为“桶”，若采用双向链表，便于实现删除操作）。采用链地址法处理冲突的散列表简称为**拉链散列表**，存储结构定义如下。其中，函数`InitChainHT`构造并返回空的拉链散列表，向量的长度为`hashsize[sizeIdx]`；`hashsize`是一个可根据需求预设的递增素数表，`sizeIdx`保存了当前长度的素数索引，在扩容重建时，可利用它取得下一个素数作为新的向量长度。

清華大學出版社 TSINGHUA UNIVERSITY PRESS

拉链散列表类型定义

```
int hashsize[] = {7, 13, 23, 47, 83, 127,
                  251, 509, 1021, 2039, ... };
typedef struct {
   int     key;        // 关键字
   char   *s;          // 元素信息，此设为C串
} ElemType;            // 元素类型
typedef struct LNode {
   ElemType         data;
   struct LNode   *next;
} LNode, *LinkList;     // 单链表类型
typedef struct {
   LinkList *lists;    // 链表指针向量
   int       size;     // 向量长度
   int       sizeIdx;  // 向量长度索引
   int       count;    // 表中元素个数
} *ChainHT;            // 拉链散列表指针类型
```

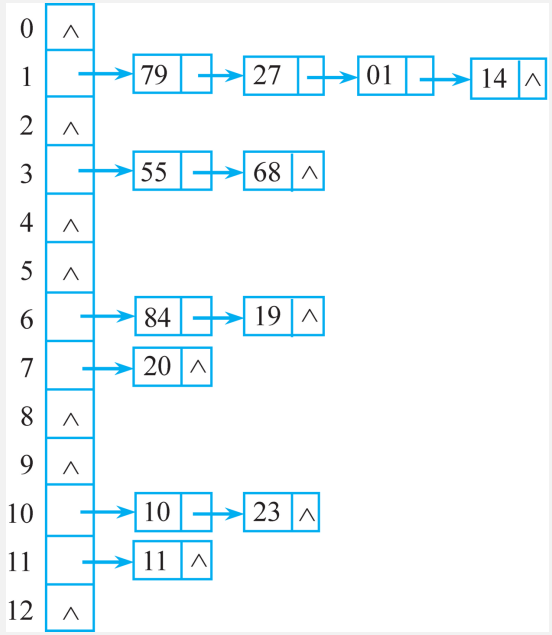


图9.37　链地址法处理冲突时的散列表

对拉链散列表的构建、查找和插入算法的学习，可在AnyviewC进行可视交互跟踪观察树形的动态变化

对冲突较多的大数据应用，拉链法的桶可以变型为用红黑树或跳跃表替代链表。如果链表长度很小(比如10以内)，用链表作为桶仍更简单有效。



2. 开放定址法

开放定址法是在散列地址区间内探查可用的空位置。在处理散列地址冲突时，若得到的下一个散列地址H_i仍然发生冲突，则再求下一个地址H_{i+1}。依次类推，直到H_k不发生冲突为止，H_k即为元素在表中的地址。

$$H_i = (H(key) + d_i) \text{ MOD } m \quad (i=1,2,\ldots,k,\ k \le m-1) \qquad (9\text{-}25)$$

其中，$H(key)$为散列函数；m为散列表容量；d_i为地址增量序列，常用下列三种：

① $d_i = 1,2,\ldots,m-1$，称为线性探查增量序列；

② $d_i = 1^2,-1^2,2^2,-2^2,\ldots,\pm k^2$（$k \le m/2$），称为二次探查增量序列；

③ $d_i =$ 伪随机数序列，称为伪随机探查增量序列。

采用开放定址法处理冲突的散列表称为**开放定址散列表**或者**分散表**，存储结构定义如下，与拉链法不同，向量单元是元素类型，并增设两个域`hi`（当前散列地址）和`lf`（装载因子）。初始化可指定素数表`hashsize[]`中某个值作为向量长度。一旦散列表的元素个数超过容量与装载因子的乘积，就取`hashsize[]`中下一个素数作为向量长度重建散列表。

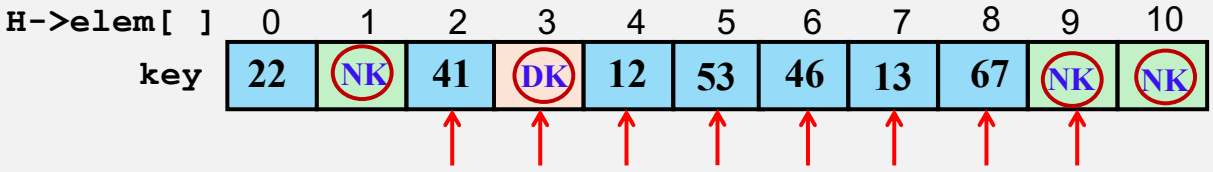
算法9.35 开放定址散列表的查找

清華大學出版社 TSINGHUA UNIVERSITY PRESS

示例 1. 查找12过程：计算散列值，沿着探测地址顺序查找

2. 查找关键字30

查找失败时，p的值就是关键字30插入的位置



DK: 已删除键

NK: “空”键

H 30

```
Status Search(OpenHT HT, KeyType K) { // 算法9.35
  int hi = Hash(HT, K);   HT->i = 0;  // 求得散列地址
  while ((H->es[hi].key!=NULLKEY && !EQ(K, H->es[hi].key))
    if ((hi = Probe(HT)) < 0) return UNSUCCESS; // 探测，若失败则返回
  if (EQ(K, HT->es[hi].key))
    return SUCCESS;
  else return UNSUCCESS;
}
```

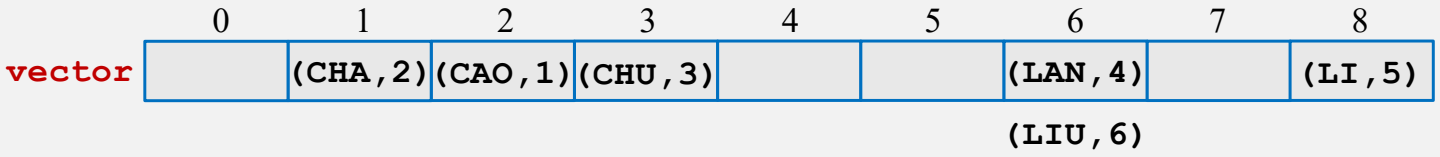
```
int Hash(OpenHT HT, int K) {
  return HT->hi = k % HT->size;
}   // 哈希函数
```

```
void Probe(OpenHT HT) {   // 线性探查函数
   HT->i++;
   return (HT->hi = (HT->hi+1) % HT->size);
}
```

布谷鸟散列——单向量+多散列函数

布谷鸟散列（cuckoo hashing,Pagh和Rodler 2004) 模拟布谷鸟占巢行为，有多种变型。

这里用示例介绍“单向量+双散列函数”形式的布谷鸟散列表。



从空表开始，依次无冲突插入(CAO,2)、(CHA,1)、(CHU,3)、(LAN,4)

插入(LI,5)　h0=8　无冲突

插入(LIU,6)　h0=6(冲突)，　h1=2(冲突)，　随机选中　h1=2　踢走(CAO,1)

重新插入(CAO,1)　h0=2(冲突)，　h1=5　无冲突

要求课外在AnyviewC可视交互学习布谷鸟散列表的初建、插入、扩表、查找和删除算法



9.4 布隆过滤器

清華大學出版社
TSINGHUA UNIVERSITY PRESS

布隆过滤器（Bloom Filter）是一种以伯顿·霍华德·布隆（Burton Howard Bloom）的名字命名的数据结构，发明于20世纪70年代。近十余年来，面对大数据的需求，布隆过滤器得到了应用和发展。它与散列表一样快，但更节省内存。

布隆过滤器和散列表之间有4个显著差异。

（1）基本型布隆过滤器不存储数据，只作检索操作：数据在集合中吗？换言之，它实现了散列集合的API，而不是散列表的API；

（2）与散列表相比，布隆过滤器需要更少的内存，是应用于大数据集的主要原因；

（3）虽然得到的阴性答案有100%的准确率，但也可能存在假阳性的误报。这将在稍后予以解释。在此，请先记住布隆过滤器有可能在某个值还没有被添加的时候就回答它是否已被添加；

（4）无法从基本型布隆过滤器中删除值[1]。

布隆过滤器属于概率型数据结构，有必要在其准确率与其使用的内存之间进行权衡。使用的内存越少，布隆过滤器返回的**假阳性**（不存在的元素被误报存在）就越多。给定了需要存储的值的数量之后，有一个确切公式可以用来计算将假阳性率保持在某个阈值之下所需的内存量，可用于其构建时的内存分配。

位向量，这是实现布隆过滤器的基础。

布隆过滤器工作原理

一个布隆过滤器BF(m,k)由两部分组成：

（1）一个包含m个元素的数组；

（2）一个包含k个散列函数的集合。

对于大数据集，这个数组通常是位向量，初始所有位都被置0。所有散列函数都会输出一个介于0和m-1之间的索引。

由此可知，数组元素和添加到布隆过滤器的键（关键字简称为键）之间没有任何一一对应的关系。将用到k位（也就是数组里的k个元素）来存储布隆过滤器中的每个键。这里的k通常比m小得多。

k是在创建数据结构时选择的常量，因此添加的每个键都会使用相同大小的内存来存储，也就是k位。对于字符串来说，这是非常优秀的存储方案，因为它可以使用恒定的内存大小（仅k位）来将任意长度的字符串表示（添加）在过滤器中。

如果在过滤器插入一个新键，就需要计算数组的k个索引，分别由值 h_0(key)~$h_{(k-1)}$(key)表示，并将这些位设置为1。

如果需要查找一个键，则仍然需要像插入操作那样计算它的k个散列值，不过这里只需要检查散列函数返回的索引处的那k位。另外，当且仅当所有位都被设置为1时，才返回TRUE。

布隆过滤器工作原理 —— 示例

清華大學出版社 TSINGHUA UNIVERSITY PRESS

例9-15　布隆过滤器BF(17,3)的插入和查找示例。

初始过滤器为空，是图9.41(a)的长度为17的全零位数组。

(b)是插入键x_1和x_2后的状态，它们由3个散列函数求得的索引三元组分别为(1,7,12)和(7,10,14)，数组中对应的元素被置1。显然，7是两个三元组的重叠索引。

(c)是查找键y_1、y_2和y_3的状况，其索引三元组分别为(1,5,13)、(7,10,14)和(10,12,14)；读取相应的位，y_1只有1个位是1，可判其不存在；而y_2的3个位均为1，可判其在集合中；y_3的3个位也均为1，但是，不能排除过滤器返回的是假阳性。

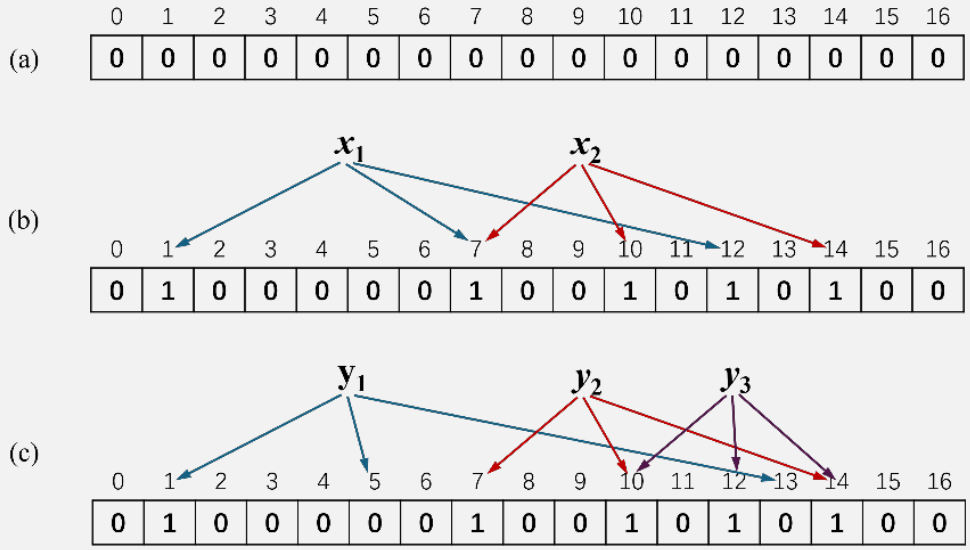


图9.41 布隆过滤器示例

散列函数解决方案

在理想情况下，需要k个不同的独立散列函数，以保证对于相同的值不会出现两个索引重复的情况。不过，设计大量独立的散列函数并不容易。常用的解决方案有以下几种：

（1）使用一个带参数的元函数$H(i)$：这个元函数是散列函数的生成器，旨在接收初始值i并输出散列函数$H_i=H(i)$。可通过在 k 个不同的（通常是随机的）值上调用生成器 H来创建k个不同的数列函数H_0~H_{k-1}。

（2）使用单个散列函数H，但初始化k个随机（且唯一）值的列表L。对于要被插入或搜索的每个键key，可通过将L[i]添加或附加到key来创建出k个不同的值，然后使用H对它们进行散列。（精心设计的散列函数会因为输入的微小变化而产生完全不同的结果）

（3）使用双重或三重散列①。

虽然最后这个解决方案并不能保证生成的散列函数之间的独立性，但足以证明②即使放宽这个约束也不会让假阳性率增加太多。为了简单起见，可使用带有两个独立散列函数的双重散列：Murmur 散列和Fowler-Noll-Vo（fnv1）散列。

因此，对于介于0和k-1之间的i，实现中的第i个散列函数的通用公式为

$$h_i(\text{key}) = \text{murmurhash(key)} + i*\text{fnv1(key)}+i*i$$



9.4.3 布隆过滤器基本型的实现

基本型布隆过滤器的集合API的实现。

1. 散列函数集合的构成

可对9.3.3节布谷鸟散列函数进行改进，引入伪随机数列hashi[0..k-1]，用hashi[i]替代i，构成以下k个双重散列函数。

```
int Hash(int i, BloomFilter bf, KeyType key) {
    // 参数化的第i个散列函数（i=0..k-1），bf-M为过滤器的总位数，bf->hashi是伪随机数列
    int h = (HashFunc1(key) + bf->hashi[i] *
             HashFunc2(key) + bf->hashi[i] * bf->hashi[i]) % bf->M;
    return  h >= 0 ? h : h + bf->M;     // 避免返回负值
}
```

散列函数集合是通过使用双重散列并以k种不同方式组合两个参数构成的。相对于线性或二次散列来说，双重散列增加了可以获得的散列函数。散列函数的数量可从$O(k)$增长到$O(k^2)$。尽管如此，这与均匀散列所要保证的理想状态$O(k!)$仍然相去甚远。但在实际工作中，这已经足够，并且能够保持良好的性能（这意味着更低的碰撞率）。为了增加随机性，还可以用伪随机数列替换双重散列函数中的i值。

9.4.3 布隆过滤器基本型的实现（续）

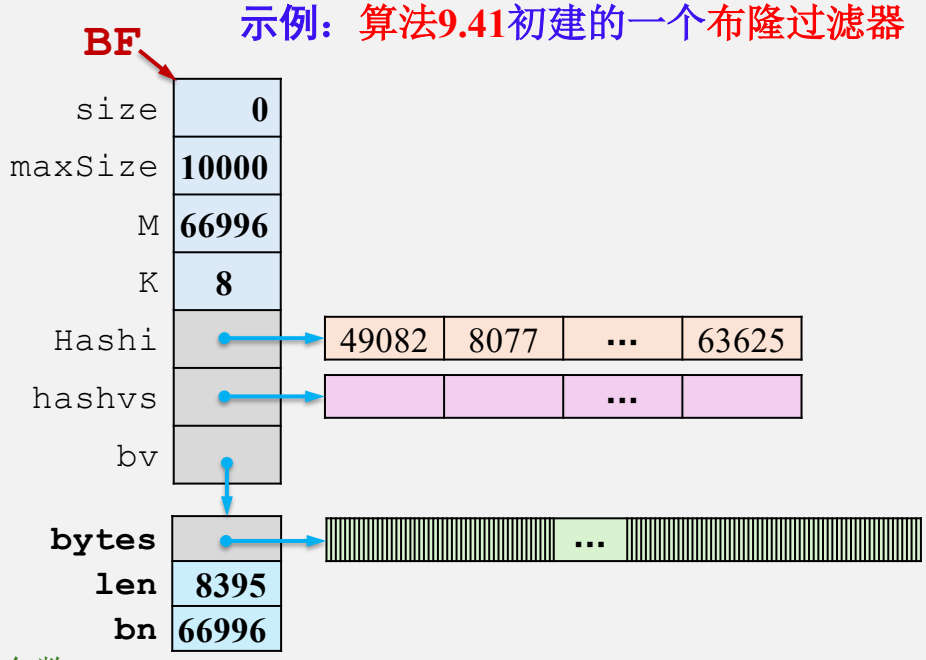
2. 存储结构和构造函数

以含位向量BitsVector和散列函数相关参数域的结构体作为基本型布隆过滤器的存储结构。

构造函数的主要任务是设置布隆过滤器的所有内部状态，确定需要分配的资源，从而使存储容量能够满足预定准确率。

```
typedef struct {
    int    size;       // 键实际个数
    int    maxSize;    // 键最大个数
    int    M;          // 容量(二进位的个数)
    int    K;          // 初始化确定的散列函数个数
    int   *hashi;      // 存储初始化时为各散列函数生成的随机参数
    int   *hashvs;     // 存储最新求得K个散列值
    BitsVector  bv;    // 位向量
} *BloomFilter;        // 布隆过滤器指针类型
```

示例：算法9.41初建的一个布隆过滤器

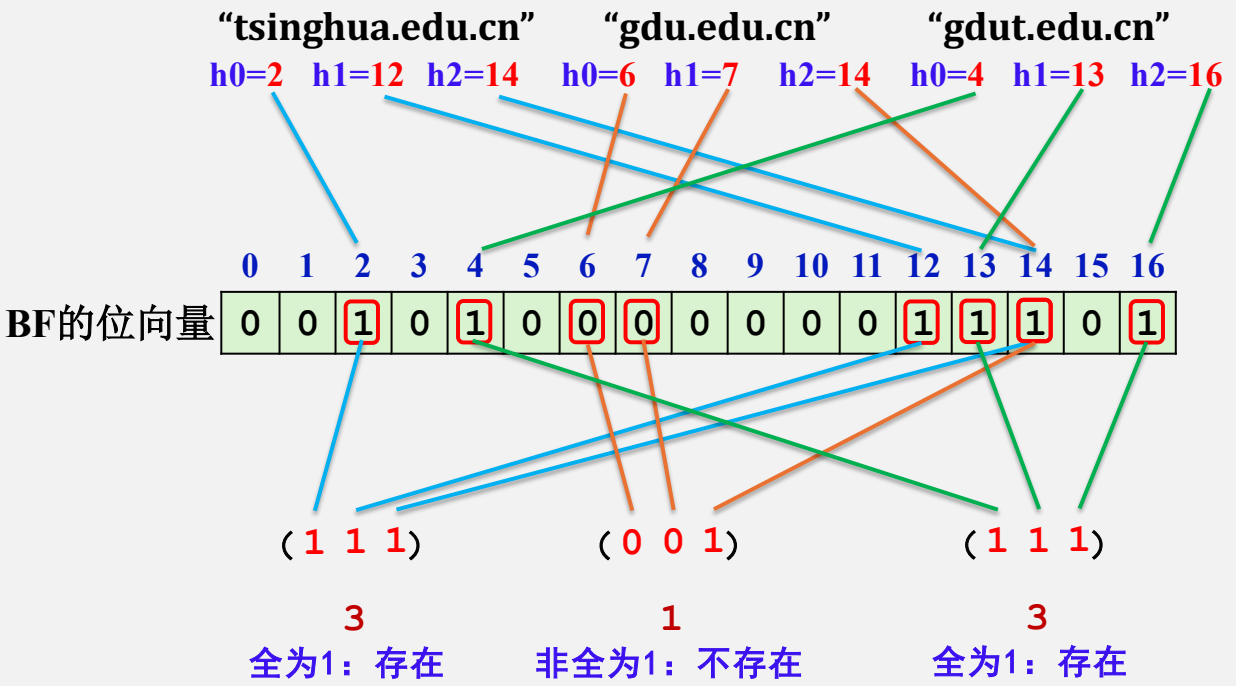


清華大學出版社 TSINGHUA UNIVERSITY PRESS

3. 查找关键字

假设关键字是字符串，但也可以是任何其他的可序列化对象。通过使用位向量，查找布隆过滤器中的关键字变得非常简单。只需要检索那些会被用来存储关键字的位的位置，并检查这些位是否都是1即可。

图9.42展示了三个键的查找过程。查找操作的实现如**算法9.42**所示。函数ContainsBF接收一个键到参数key，当且仅当与键对应的所有位都被设置为1时才返回TRUE。



清華大學出版社 TSINGHUA UNIVERSITY PRESS

4. 存储关键字

存储键与查找键非常相似，但另外需要一些步骤来跟踪添加到过滤器中的元素数量，并且使用的函数是SetBV而不是GetBV。

算法9.43实现了插入操作，当计算过滤器的大小时，需要跟踪添加到过滤器中的不同键的数量，而不是跟踪Insert方法的总调用次数。因此，需先确认键key不存在，才实施置位，避免size域的重复计数。size域对于估计过滤器的假阳性率至关重要，因此需要准确计算实际存储的元素。

